ETH zürich

Berkeley
UNIVERSITY OF CALIFORNIA

Master Thesis

TWIX 2: a framework for reliable data extraction at scale from templatized documents

Riccardo Bollati

First author, ETH Zurich

Yiming Lin

External supervisor, University of California, Berkeley

Prof. Gustavo Alonso

Internal supervisor, ETH Zurich

21 October 2026

Abstract

Templatized PDF documents reuse a recurring arrangement of fields, tables, or form sections across records or pages, while the associated values change. Examples include invoices, registers, and product catalogues. Extracting their contents into structured data makes large collections accessible to quantitative analysis, unlocking information otherwise confined to a format designed for human inspection. This remains challenging when documents contain variations in tables, key-value sections, or record boundaries. Direct extraction with large language models can address complex layouts, but its inference cost grows with document volume. To address this problem, model reasoning can be used to construct an extractor for a specific PDF, a reusable program which can then process additional pages without further model calls.

The original TWIX is a framework for extracting structured data from documents that share a recurring visual template. It identifies recurring field labels and their spatial relationships, organizes them into an explicit template, and uses that template to extract values and assemble related tables and key-value sections into logical records. This thesis extends that methodology through TWIX 2. The framework first uses clustering to select structurally diverse pages, it then uses language models to construct reusable extraction programs from this small, informative sample, expands the template representation to handle variations in layouts and leverages the Divide-and-conquer paradigm. This combination is intended to improve generalization by exposing relevant layout variations while narrowing each code-generation task. Explicit templates and intermediate outputs support inspection of omissions and failures.

TWIX 2 using DeepSeek V4.1 Flash was tested against a GPT-6 Sol monolithic code-generation baseline using 23 PDFs containing repeated structures and template variations. The results show higher mean test accuracy for TWIX 2 (98.72%) compared to the baseline (89.05%), better generalization to held-out pages and earlier cost recovery with a median break-even of 5.51 pages for TWIX 2 and 18.00 pages for the baseline, excluding reference-generation and scoring costs. Across the benchmark, direct LLM extraction would cost an estimated $242.90 to extract all 23 PDFs, compared with pipeline construction costs of $22.31 for the GPT-6 Sol baseline and $8.61 for TWIX 2. These figures illustrate the savings enabled by reusable extraction code and the further cost reduction achieved by the proposed framework. TWIX 2 can also use smaller models while keeping high accuracy. For example, using Qwen3.8 27B for code generation achieves 97.31% mean test accuracy, allowing for different setups (e.g. local hosting).

This thesis establishes TWIX 2 as a promising solution for accurate, transparent, and economical extraction at scale.

Acknowledgements

I would like to thank Yiming Lin for his dedicated guidance, insightful feedback, and continuous support throughout this thesis. His time and expertise were invaluable to the development of this work. I also thank Professor Gustavo Alonso for his supervision, support, and the opportunity to pursue this research.

Abbreviations

2D	Two-dimensional.
AI	Artificial intelligence.
API	Application programming interface.
AWE	Assets & Wealth Extraction benchmark.
`bid`	Block index within a record; a template attribute describing block positions.
CLI	Command-line interface.
CSV	Comma-separated values.
ETH	Eidgenössische Technische Hochschule (Swiss Federal Institute of Technology).
ETL	Extract, transform, load, as in DocETL.
F1	Harmonic-mean score combining precision and recall.
GPT	Generative pretrained transformer.
ID	Identifier.
JSON	JavaScript Object Notation.
K	Key row: a table-header row in the template-inference prompt.
KV	Key-value pair or block; also the label for a key-value row.
LLM	Large language model.
M	Metadata row in the template-inference prompt.
MDS	Multidimensional scaling.
OCR	Optical character recognition; also the implementation's name for extracted words and layout coordinates.
PDF	Portable Document Format.
QA	Quality assurance.
regex	Regular expression.
RQ	Research question.
SWE	Software engineering, as in SWE-bench and DeepSWE.
TWIX	Original template-based document extraction framework.
TWIX 2	Agentic extension of TWIX developed in this thesis.
`uid`	Unique identifier assigned to a physical block.
USD	United States dollar.
V	Value row: a table-data row in the template-inference prompt.
XOR	Exclusive OR; a logical slot representing mutually exclusive template alternatives.

Contents

1 Introduction **1**

1.1 Extracting data at scale 1

1.1.1 Templatized documents 1

1.1.2 Document structure and block types 2

1.1.3 Document families with structural alternatives 3

1.2 Background 3

1.3 Project objective 5

2 Methodology **7**

2.1 TWIX 2 Framework 7

2.2 Setting up the Environment 8

2.3 Clustering for informative page selection 9

2.3.1 Variation informed sampling 9

2.3.2 Table and key-value similarity 9

2.3.3 Forming clusters and choosing representatives 10

2.3.4 Training pages selection 11

2.3.5 Coverage limits 12

2.4 Template inference and XOR alternatives 12

2.4.1 An explicit structural contract 12

2.4.2 Exclusive-choice slots 13

2.5 Generation of physical block separation 14

2.6 Block samples and extractors 15

2.6.1 Learning one physical node at a time 15

2.6.2 Interfaces and generation validation 15

2.7 Portable execution and deterministic record grouping . . . 16
2.8 Training evaluation and bounded repair . . . 17
2.9 Cost and scaling . . . 17
2.10 The artifacts available to the user . . . 18
2.10.1 Coverage as a failure signal . . . 18

3 Experiment design **20**
3.1 Document families and data splits . . . 20
3.2 Baseline and other comparison . . . 21
3.3 Evaluation . . . 22
3.3.1 Reference generation and interpretation . . . 22
3.3.2 Scoring and interpretation . . . 22
3.3.3 Independent baseline construction and adaptation . . . 23

4 Results and Discussion **24**
4.1 Comparison with frontier models . . . 24
4.1.1 Performance mismatch analysis . . . 25
4.1.2 Cost and break-even analysis . . . 27
4.2 Paired comparison with the original TWIX . . . 27
4.3 Comparison with Qwen 27B . . . 29

5 Conclusion **32**
5.1 Future work . . . 33

Bibliography **35**

Appendix **36**

List of Figures

1.1 A full source page from the complaint register `id_11`, illustrating a templatized document. Three records repeat the same field labels and section arrangement while their values and row counts vary. Colors and redactions are present in the source document. 2

2.1 Overview of TWIX 2. Pipeline construction uses sampled evidence and staged code generation. Repair is an optional extension and was disabled in the experiments; test evaluation does not feed construction. 8

2.2 Table clustering for `id_23` (23 source pages). Top: a 2D MDS view of page dissimilarities (left) and header/structure dissimilarities relative to page 13 (right). Stars mark representatives. Bottom: representative pages 8, 13, and 20, with table borders in red and header evidence in blue. 11

2.3 The same clustering views for `id_17` (35 source pages, five clusters). The header/structure plot uses page 33 as its reference. Below are representative pages 20, 19, 4, 7, and 9 in cluster order, with the same structural overlay colors as in figure 2.2. 12

2.4 Exported template trees: four ordered physical blocks in `id_11`, and a key-value block followed by an XOR slot in `id_23`. Dashed branches link alternative table nodes 2, 3, and 4 at `bid=[1]`. Blue denotes tables, green key-value blocks, and orange the virtual XOR slot. Block descriptions are abbreviated. 14

2.5 Document excerpts matching figure 2.4: (a) one complete `id_11` record, containing nodes 0 to 3; (b), (c), and (d) the tops of `id_23` pages 8, 13, and 20. These show the header and unit variations represented by alternative table nodes 2, 3, and 4. 14

2.6 Decomposition of one `id_11` record from source page 1. Colored boundaries follow the four separator-assigned blocks, with a small padding for display. Matching arrows connect each region to its isolated crop: complaint overview (table 0), complainant details (key-value block 1), complaint details (table 2), and officer details (table 3). Each physical template node has its own extraction script. 16

2.7 Sonar coverage for the first 100 source pages of `id_22`. On source page 90, only 83 of 292 words are assigned to blocks (28.4% coverage), compared with a scan mean of 82.3%. Inspection reveals a flange-dimensions table whose schema is absent from the inferred template, illustrating how sonar directs attention to potentially missed structured content. 19

4.1 Baseline versus TWIX 2 test accuracy, ordered by document structure. Pastel green denotes a fixed page structure (10 PDFs), yellow denotes variable repetition or nesting within one template (10 PDFs), and red denotes template variations across pages (3 PDFs). Changes in row counts or values alone do not establish a new template. The mean gives equal weight to all 23 PDFs . . 25

4.2 Break-even page counts for the baseline and TWIX 2, with the same document order, colors, and structural groups as figure 4.1. The separate group on the right shows the median for each system over all 23 documents. Lower is better; the vertical axis is logarithmic. 28

4.3 Mean model cost per page across 23 documents. Dark blue: amortized TWIX 2 generation cost; shading: 10th to 90th percentiles across documents. Dashed orange: direct LLM extraction (USD 0.0492/page). The mean curves cross at approximately 7.6 pages. Costs exclude local execution; see appendix C. 28

4.4 Per-document F1 for the original TWIX extractor with agent-generated template and metadata, compared with full TWIX 2, on the same 440 pages and reference pairs. Document order and structural-group backgrounds follow figure 4.1. The separate group on the right reports the macro mean across all 23 PDFs. Scores use exact unique key-value pairs. 30

4.5 TWIX 2 test accuracy with DeepSeek V4.1 Flash and Qwen 27B across 23 documents. The rightmost group shows document-level macro means for both TWIX 2 configurations and the GPT-6 Sol baseline; baseline bars are included only in this aggregate group. Structural groups follow figure 4.1. 31

List of Tables

3.1 Training and test page budgets for the DeepSeek V4.1 Flash runs. Training-page counts match the monolithic baseline for every document. 21

4.1 Recorded document-level macro accuracy on 23 PDFs. Gap is mean training minus test accuracy, computed before rounding. 24

4.2 Document-level macro scores on 23 PDFs and 440 test pages. The agent package supplies a replacement template and metadata to the original extraction algorithm. Failed runs count as empty predictions. 29

B.1 Saved outcomes for the six baseline programs that fail on their complete PDFs. Empty-page counts concern their selected evaluation pages. A failure elsewhere in a PDF does not automatically lower the selected-page score. 66

C.1 Generation-only construction costs and continuous break-even page counts, excluding reference generation and scoring. An asterisk denotes the legacy generation-price fallback. For `id_06` and `id_14`, c_d uses separately supplied full-PDF costs. 72

Chapter 1

Introduction

1.1 Extracting data at scale

Over many years, companies have accumulated large corpora of business data in PDF documents, including invoices, forms, tables, and reports. These documents were designed primarily for human inspection: they present information in a form that people can read and interpret. Although the information may be valuable for quantitative analysis, it remains embedded in document layouts rather than readily available as structured records that can be queried, compared, or aggregated. Recovering those records is necessary to turn these archives into usable analytical datasets.

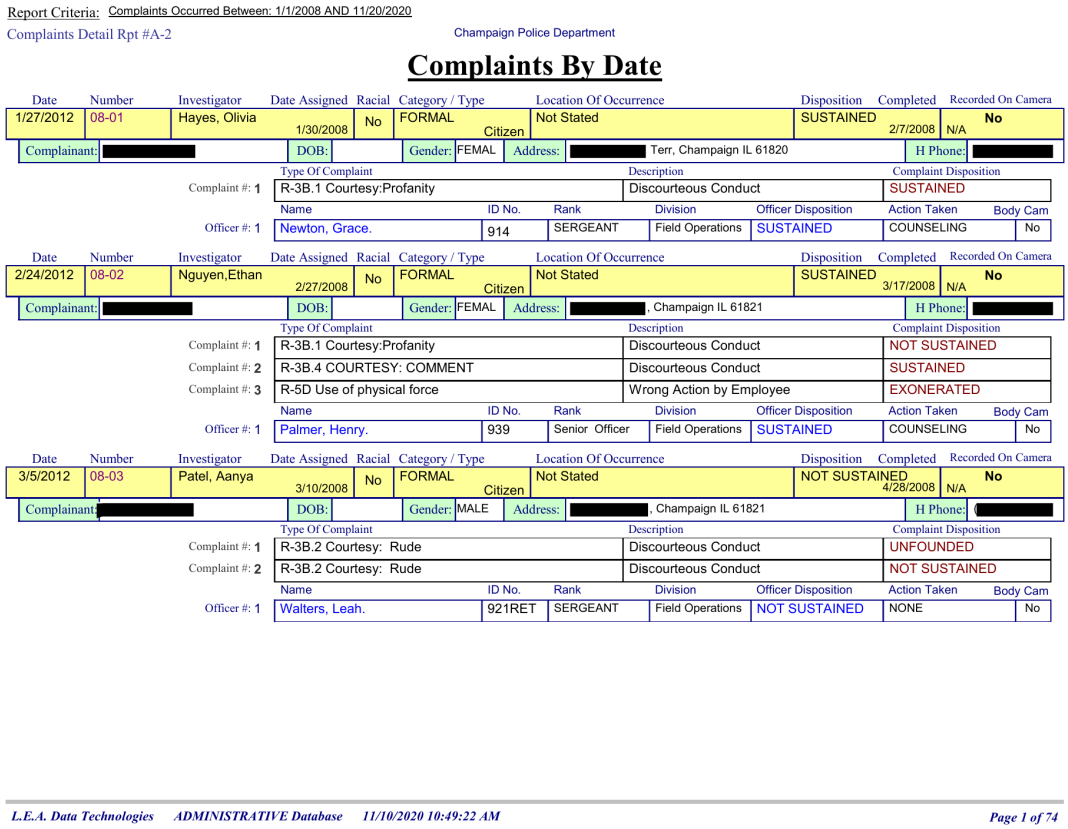
The requirement for structured output is central to this problem. Recognizing the words on a page is insufficient: an extraction system must also determine which values belong to which fields, rows, and records. In documents intended for human readers, these relationships are often left implicit and conveyed through alignment, headers, whitespace, and visual grouping. A reader interprets them from the layout. Naive text extraction, whether from an existing PDF text layer or through optical character recognition (OCR), does not by itself reconstruct these relationships. Producing structured data therefore requires processing and interpreting the document structure as well as recovering its text.

1.1.1 Templatized documents

A templatized document follows a recurring visual pattern for presenting the same kinds of information. We call this pattern a template: it specifies which fields and sections recur and how they are arranged, while the values vary between instances. For example, invoices may reuse a supplier-details section and an item table, and a complaint register may repeat an overview, personal details, and tables describing each case. The repetition can occur within one page, across pages, or across several files from the same document family.

The complaint register in figure 1.1 illustrates this idea. Three records share the same se-

quence of overview fields, complainant details, complaint entries, and officer details. Their values and the number of complaint rows differ, while the recurring field labels and organization make the records recognizable to a reader.



Report Criteria: Complaints Occurred Between: 1/1/2008 AND 11/20/2020

Complaints Detail Rpt #A-2 — Champaign Police Department

Complaints By Date

Date	Number	Investigator	Date Assigned	Racial	Category / Type	Location Of Occurrence	Disposition	Completed	Recorded On Camera
1/27/2012	08-01	Hayes, Olivia	1/30/2008	No	FORMAL Citizen	Not Stated	SUSTAINED	2/7/2008	No

Complainant: [redacted] DOB: Gender: FEMAL Address: [redacted] Terr, Champaign IL 61820 H Phone: [redacted]

	Type Of Complaint	Description	Complaint Disposition
Complaint #: 1	R-3B.1 Courtesy:Profanity	Discourteous Conduct	SUSTAINED

	Name	ID No.	Rank	Division	Officer Disposition	Action Taken	Body Cam
Officer #: 1	Newton, Grace.	914	SERGEANT	Field Operations	SUSTAINED	COUNSELING	No

Date	Number	Investigator	Date Assigned	Racial	Category / Type	Location Of Occurrence	Disposition	Completed	Recorded On Camera
2/24/2012	08-02	Nguyen,Ethan	2/27/2008	No	FORMAL Citizen	Not Stated	SUSTAINED	3/17/2008	No

Complainant: [redacted] DOB: Gender: FEMAL Address: [redacted], Champaign IL 61821 H Phone: [redacted]

	Type Of Complaint	Description	Complaint Disposition
Complaint #: 1	R-3B.1 Courtesy:Profanity	Discourteous Conduct	NOT SUSTAINED
Complaint #: 2	R-3B.4 COURTESY: COMMENT	Discourteous Conduct	SUSTAINED
Complaint #: 3	R-5D Use of physical force	Wrong Action by Employee	EXONERATED

	Name	ID No.	Rank	Division	Officer Disposition	Action Taken	Body Cam
Officer #: 1	Palmer, Henry.	939	Senior Officer	Field Operations	SUSTAINED	COUNSELING	No

Date	Number	Investigator	Date Assigned	Racial	Category / Type	Location Of Occurrence	Disposition	Completed	Recorded On Camera
3/5/2012	08-03	Patel, Aanya	3/10/2008	No	FORMAL Citizen	Not Stated	NOT SUSTAINED	4/28/2008	No

Complainant: [redacted] DOB: Gender: MALE Address: [redacted], Champaign IL 61821 H Phone: [redacted]

	Type Of Complaint	Description	Complaint Disposition
Complaint #: 1	R-3B.2 Courtesy: Rude	Discourteous Conduct	UNFOUNDED
Complaint #: 2	R-3B.2 Courtesy: Rude	Discourteous Conduct	NOT SUSTAINED

	Name	ID No.	Rank	Division	Officer Disposition	Action Taken	Body Cam
Officer #: 1	Walters, Leah.	921RET	SERGEANT	Field Operations	NOT SUSTAINED	NONE	No

L.E.A. Data Technologies *ADMINISTRATIVE Database* *11/10/2020 10:49:22 AM* *Page 1 of 74*

Figure 1.1: A full source page from the complaint register `id_11`, illustrating a templatized document. Three records repeat the same field labels and section arrangement while their values and row counts vary. Colors and redactions are present in the source document.

1.1.2 Document structure and block types

To extract structured data, the system must identify both individual fields and the larger units to which they belong. A field is a named attribute, such as a date, complaint number, or address; its value is the content printed for that attribute. A schema specifies the fields and their organization. We use the term block for a visually coherent region containing structured content that can be extracted as one unit. A block may be recognized through headers, alignment, spacing, or borders; a visible rectangular border is not required.

We consider two physical block types. A table block organizes values into rows under a shared set of column headers. The headers identify the fields, and the rows provide repeated observations under those fields. A table may contain one or many rows, wrapped text, or several header tiers; these variations do not by themselves change its block type.

A key-value block presents explicitly labelled attributes and their associated values, rather than repeated observations under shared column headers. Labels and values may appear beside one another, below one another, or across several lines. Tables and key-value blocks can occur together within the same record. Blocks of the same broad type may have different schemas: two tables can contain different fields and require different extraction routines.

A physical block is one occurrence of such a region on a page. A physical template node describes a recurring block schema, allowing several block occurrences to use the same extraction routine. Each physical block contains words from only one page; a table continuing onto another page produces a separate continuation block. This is what we mean by page-local processing.

A logical record collects the blocks that describe one case or entity. In the complaint register, the overview, complainant details, complaint table, and officer table together describe one case. The three cases shown on the example page therefore form three records, each composed of several blocks. More generally, a record may contain repeated blocks or extend across pages. A template describes the recurring organization of these blocks, while record grouping associates their detected occurrences with the appropriate case.

1.1.3 Document families with structural alternatives

Let $\mathcal{D}$ be a family of PDFs generated from a related set of visual layouts. Given a selected training sample S, the construction procedure learns a template T and a program bundle P. For a new document $D \in \mathcal{D}$, the runtime computes

$$\widehat{Y}(D) = P\big(\mathrm{Words}(D), T\big), \tag{1.1}$$

where $\widehat{Y}$ contains extracted physical blocks and grouped logical records. The target is faithful recovery of visible structured content, rather than open-ended interpretation or summarization.

This is a within-family generalization problem. The same program is not expected to process arbitrary PDFs with unrelated layouts. Even within a family, a genuinely new schema may require additional evidence and a revised pipeline.

1.2 Background

Vision-Language Models for Document Data Extraction

A recent benchmark published by Mantle, a software company working with equity and private fund data, provides evidence of what frontier multimodal models can achieve in document extraction. Its Assets & Wealth Extraction (AWE) benchmark evaluates the extraction of names, dates, and numbers from 78 synthetic financial PDFs, with 7,657 fields

scored for each model. The six highest-scoring models: Claude Opus 5.5, Claude Fable 5.1, GPT-6 Astra, GPT-6.1 Sol, GPT-5.6 Sol, and Gemini 4 Argon achieved field-level accuracies between 94.1% and 95.3%, averaging 94.8%. These results show strong performance on the documents and fields tested, though they do not establish error-free extraction [7].

These results motivate seeking comparable extraction quality through reusable programs that avoid repeated model inference. The present thesis evaluates this goal through agreement with AI-generated references on its own document collection; its scores are not directly comparable with AWEs field-level accuracies.

TWIX and Visual template inference

Lin et al. describe TWIX as a framework that infers recurring fields from location patterns and organizes them into an explicit template using visual constraints. The template guides predefined extraction algorithms, allowing additional pages to be processed without repeated model calls. Its reliability, however, depends on the quality of the text and geometry recovered from the PDF. Noise in this representation can include fragmented headers, incorrectly ordered phrases, imprecise bounding boxes, and page headers or footers mixed with structured content. These disturbances can obscure recurring field patterns or create misleading alignments, causing genuine fields to be missed and unrelated text to be interpreted as data. Such errors can propagate from field inference to template construction and subsequent extraction. For example, a header split across several lines may be interpreted as separate fields or values, while a wrapped cell may become multiple apparent rows. TWIX tolerates some bounding-box inaccuracies and consistent recognition errors, but semantic correction and reconstruction of multi-line phrases lie outside its core scope.

Language models as program generators

Recent benchmarks evaluate coding agents on repository-level software engineering tasks. SWE-bench assesses systems on real GitHub issues that require modifying existing codebases, with solutions evaluated through executable tests [2]. More recently another benchmark, DeepSWE introduces original, long-horizon engineering tasks and hand-written functional verifiers that assess the requested behavior rather than a particular implementation [3]. These benchmarks motivate investigating language models as generators of dedicated extraction programs for specific document families.

Other systems based on reusable extraction code

Another system to perform data extraction is Evaporate, a pipeline that focuses on semi-structured data, including PDFs. This pipeline leverages direct model extraction and model-generated extraction code [4].

DocETL, introduced by Shankar et al., provides a declarative framework for complex document processing using LLM-powered operations [5]. Its agentic optimizer rewrites pipelines by dividing documents into smaller chunks, decomposing complex tasks into simpler operations, and evaluating candidate plans through task-specific validation prompts. DocETLs semantic operations invoke LLMs during execution, so inference costs continue to grow with the volume of data processed. Together, these systems motivate reusable computation and task decomposition.

1.3 Project objective

Documents can encode a database distributed across a large number of pages and files. This scale makes extraction cost a practical constraint. Language models can interpret document layouts, but applying them directly to every page repeats model inference throughout the collection. For sufficiently large corpora, the accumulated inference cost can make this approach economically infeasible. A scalable approach must therefore recover structure reliably while limiting repeated interpretation and development effort.

Recurring structure offers an opportunity to construct an extraction program from a small sample and reuse it across the collection, but this raises four challenges. First, a limited sample may contain many examples of one layout while missing less frequent structures elsewhere in the document. The selected pages must therefore provide complementary evidence within the available budget. Second, repetition is not always exact: sections can repeat or disappear, tables can continue across pages, and alternative column layouts may require different extraction rules. Third, constructing a complete extractor combines several difficult tasks: locating relevant regions, assigning values to fields and rows, and assembling related sections into records. A program that succeeds on the sample may still encode assumptions that fail on later pages. Finally, failures can be silent: plausible structured output does not reveal whether relevant content was omitted. Reliable reuse therefore requires both handling structural diversity and providing evidence with which to inspect the extraction.

The research questions to answer are:

RQ1. How well do generated pipelines generalize from sampled training pages to the rest of the target PDF, covering recurring structures and variations within the same document family?

RQ2. Does deterministic clustering capture structural groups within a PDF and enrich the information available in a limited training budget through informed page sampling?

RQ3. Do XOR template slots successfully represent and handle template variations across pages?

The objective of this thesis is to develop reusable extraction pipelines that approach the quality of direct LLM-based document extraction while avoiding repeated model inference as document volume grows. To pursue this goal the thesis develops a framework that builds

on the principles introduced by TWIX, using the capabilities of newer language models to address limitations of the original system. In particular, it aims to support more flexible document structures, including multi-line headers and values, small template variations, and more complex layouts, while improving robustness to noise in the extracted text and geometry. A further goal is to preserve TWIXs separation between pipeline construction and extraction, allowing additional pages to be processed without further model calls and thereby decoupling document volume from recurring model-inference costs. The broader goal is to preserve the benefits of reusable, template-based extraction while expanding the range of documents that can be processed reliably at scale.

Chapter 2

Methodology

2.1 TWIX 2 Framework

The framework turns a limited set of example pages into an explicit template and a reusable extraction program. It separates construction, where models infer structure and write code, from execution, where the completed program extracts data from additional pages. Figure 2.1 summarizes these stages and their inspection artifacts.

Construction first identifies where structured content begins and scans the document using text and geometry. Independent comparisons of table and key-value layouts identify groups of structurally similar pages. The sampler selects initial examples of record order together with pages that expose different detected layouts. The models therefore receive complementary structural evidence instead of spending the sample budget on many instances of the same pattern.

From the selected pages, template inference identifies the fields and organization of recurring blocks, including alternative layouts at the same logical position. This template supplies a common structural contract for the generated programs. A separator learns to locate physical block occurrences in page text and geometry. Each specialized extractor then handles one block schema, reducing the task to recovering values from a particular table or form section. Deterministic grouping combines the resulting blocks into logical records, preserving which extracted regions belong to the same case.

Execution reuses the completed template, separator, extractors, and grouper. On each new page, the program obtains words and coordinates, identifies the applicable blocks, runs the corresponding extraction routines, and assembles the structured output. The reusable knowledge is the procedure for interpreting recurring layouts; the values are read from the current input. This execution stage requires no language-model calls.

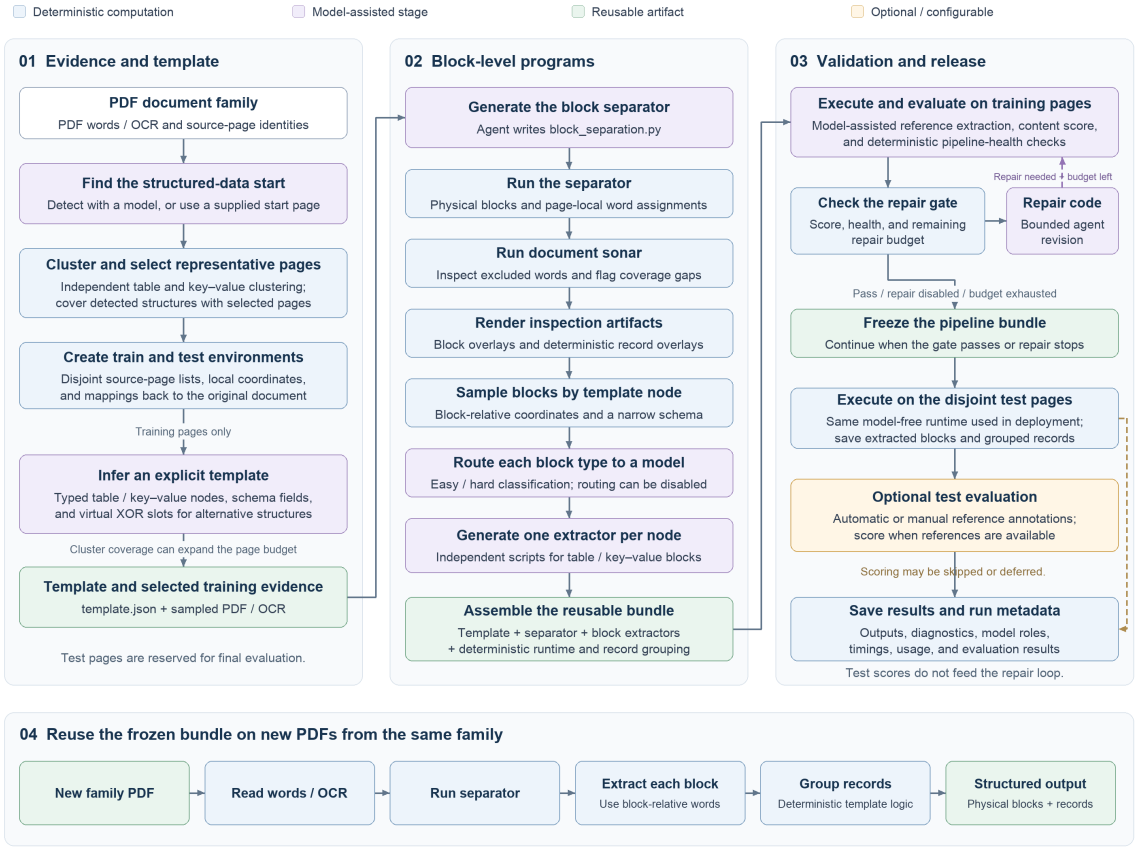


Figure 2.1: Overview of TWIX 2. Pipeline construction uses sampled evidence and staged code generation. Repair is an optional extension and was disabled in the experiments; test evaluation does not feed construction.

2.2 Setting up the Environment

The implementation uses `pdfplumber` to obtain words and their bounding boxes [6]. The repository refers to these artifacts as OCR, and this thesis adopts that naming convention where it matches the implementation.

`pdfplumber` is an open-source Python library for extracting text and layout information from PDF documents. It provides access to individual characters and words, together with bounding boxes describing their position and size on the page. This allows document content to be examined through both its text and its spatial organization.

Preprocessing converts the PDF into a shared representation of words and their positions for the later stages. Text supplies field labels and values, while coordinates preserve the alignments and spatial groupings needed to interpret them. Source-page mappings keep the selected examples and extracted regions traceable to the original document. This common representation lets clustering, block separation, and value extraction operate on consistent evidence.

Words initially receive globally unique identifiers and physical, one-based source page num-

bers. A front-matter stage identifies the start of structured content, avoiding the use of introductory narrative as the main learning evidence. A manually supplied start page bypasses that model decision. The full prompt appears in appendix A.1.

Training and test PDFs are built from explicit lists of source pages. They may be noncontiguous, but each split is renumbered locally from one. Artifacts preserve the mapping between split-local and physical source pages.

2.3 Clustering for informative page selection

The number of training pages is a useful proxy for the amount of evidence available to the models and is the measure used in this evaluation. Although approximate, it is motivated by the human-readable design of the documents considered here: page dimensions and legibility requirements place practical limits on the amount of information each page can contain.

For a nominal budget B, the objective is to select informative pages rather than spend the budget on repeated examples of an already observed structure.

2.3.1 Variation informed sampling

In a document the first few pages often reveal normal record order but may omit less frequent layouts. A contiguous sample can therefore provide many examples of one structure and no example of a later alternative. Clustering serves as a deterministic structural scan: it identifies potentially different page structures before the selected evidence reaches the models. Its purpose is to increase structural diversity within the training-page budget.

Starting from the detected beginning of structured content, the scan considers up to 500 source pages. It uses PDF words, their bounding boxes, and detected table lines without calling a language model. Table and key-value structures are clustered independently because their distinguishing features differ. Two pages may have the same tables but different key-value sections, so a page can belong to one table cluster and a different key-value cluster. This separation avoids letting a dominant table conceal a change in a smaller form. Figures 2.2 and 2.3 illustrate the detected groups and representative page structures for `id_23` and `id_17`.

2.3.2 Table and key-value similarity

Table similarity combines header evidence with physical structure. Header comparison uses overlapping column-label words and their spatial arrangement, assigning 88% weight to text and 12% to header geometry. The structural description captures normalized vertical boundary positions, horizontal rule spans, and row spacing. It emphasizes recurring column arrangements rather than raw row counts, reducing sensitivity to how many data rows appear

on a page. Where usable PDF borders are unavailable, rendered line detection or word alignment below detected headers provides alternative geometric evidence.

The geometric features are encoded as numerical vectors. The vertical-boundary, horizontal-rule, and row-spacing components are normalized separately, scaled to emphasize column structure, and concatenated into a page descriptor $\mathbf{g}_p$. Geometric similarity is computed using cosine similarity,

$$G(p, q) = \frac{\mathbf{g}_p \cdot \mathbf{g}_q}{\|\mathbf{g}_p\| \, \|\mathbf{g}_q\|}. \tag{2.1}$$

A value close to one indicates similar structural patterns, while lower values indicate different layouts. Comparing vector directions reduces dependence on the overall amount of detected line evidence; if either descriptor is empty, similarity is set to zero.

When both pages have usable borders and headers, their table similarity is

$$S_{\text{table}}(p, q) = 0.60H(p, q) + 0.40G(p, q), \tag{2.2}$$

where H is header similarity and G is geometric similarity. The weights adapt when evidence is missing; for example, comparison relies more heavily on headers when border structure is unavailable. When a page contains several tables, a change in one table should not be hidden by similarities in the others. When individual tables can be identified reliably, a substantial difference in one table keeps the pages in separate clusters.

Key-value comparison instead looks for recurring labels whose associated values vary across occurrences. Value variation helps identify data-bearing fields, but the value text itself does not enter page similarity. Candidate filtering removes likely table fragments and page-edge metadata. Each accepted group is described by its keys, their relative positions, and the offsets between labels and values, with distances normalized by local text height. Group similarity combines 65% key overlap and 35% geometry. When two pages contain several key-value groups, the system pairs the most similar groups, using each group only once. Any groups left without a match reduce the overall page similarity.

2.3.3 Forming clusters and choosing representatives

Each clustering branch computes pairwise page similarities and converts them to distances through $D(p, q) = 1 - S(p, q)$. Hierarchical clustering starts with one cluster per eligible page and progressively merges similar clusters. The selected configuration uses *complete linkage*: the distance between two clusters is the largest distance between any pair of pages across them. A page therefore cannot join a cluster merely because it resembles one member while differing strongly from another. The similarity cutoffs are 0.83 for tables and 0.77 for key-value structures, selected empirically through testing on the dataset. The number of clusters emerges from these comparisons rather than being specified in advance.

Each cluster has a representative, or *medoid*: the observed page with the smallest total distance to the other pages in that cluster. This identifies a typical example, but the final

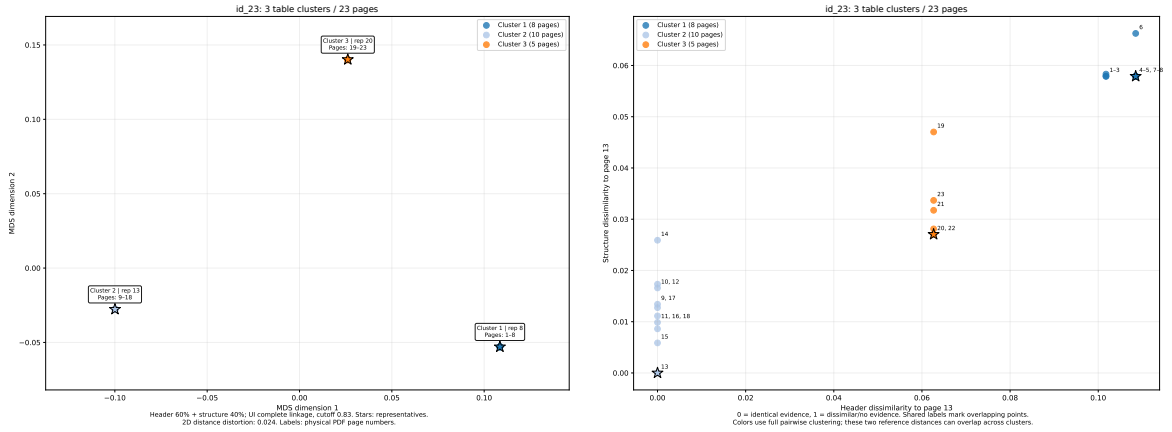
sampler can prefer another page if it covers more currently unrepresented structures.

2.3.4 Training pages selection

The sampler selects pages that collectively represent the detected table and key-value structures. When the budget allows, it begins with the first three pages containing structured data, which show the usual order of sections and how records are organized.

It then adds the page that introduces the most structures not yet represented in the sample. For example, a page containing both a new table layout and a new key-value arrangement is preferred over one introducing only the same table layout. When pages provide equal coverage, cluster representatives are preferred, followed by earlier pages.

Selection continues until all detected clusters are represented or a configured page limit is reached. If coverage requires fewer pages than requested, additional pages fill the remaining budget. By default, the requested count is a minimum, so representing every detected cluster can require more pages.



Cluster 1: page 8 **Cluster 2: page 13** **Cluster 3: page 20**

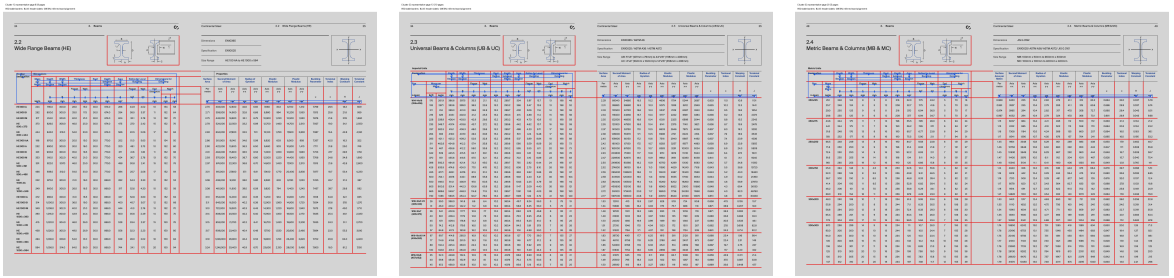


Figure 2.2: Table clustering for `id_23` (23 source pages). Top: a 2D MDS view of page dissimilarities (left) and header/structure dissimilarities relative to page 13 (right). Stars mark representatives. Bottom: representative pages 8, 13, and 20, with table borders in red and header evidence in blue.

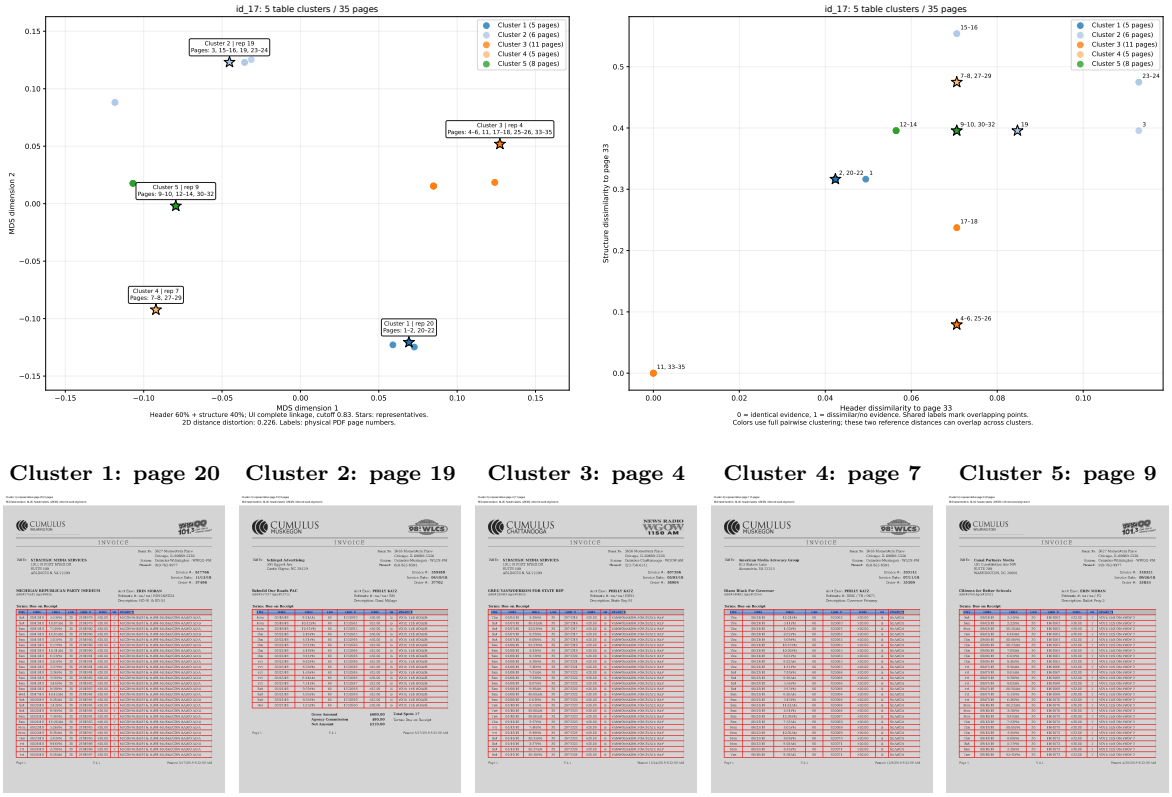


Figure 2.3: The same clustering views for `id_17` (35 source pages, five clusters). The header/structure plot uses page 33 as its reference. Below are representative pages 20, 19, 4, 7, and 9 in cluster order, with the same structural overlay colors as in figure 2.2.

2.3.5 Coverage limits

Cluster coverage is coverage of *detected* structures inside the scan horizon. A rare layout can be missed by the scanner, merged with another cluster, or appear only after page 500. Increasing model capability cannot remove an evidence gap when an unseen schema is absent from the sample.

2.4 Template inference and XOR alternatives

We use for template inference the methodology of TWIX [1], which represents recurring document structures explicitly to guide extraction and record reconstruction. TWIX 2 expands this representation with virtual XOR blocks for mutually exclusive layouts, while retaining separate schemas for the physical alternatives. LLM-based inference constructs the expanded template from representative training pages.

2.4.1 An explicit structural contract

Template inference establishes what recurring structures the generated programs must recognize and how their outputs should be organized. It receives layout-preserving text for

all selected training pages and returns an ordered set of physical table and key-value nodes together with metadata patterns. Each physical node defines the fields expected from one recurring block schema. This shared specification is the structural contract: the separator identifies occurrences of these nodes, the extractors recover their field values, and the grouper uses their relationships to assemble records. Inference therefore identifies fields and their organization rather than extracting individual values.

Metadata patterns describe framing text, such as headers and footers, that helps distinguish data regions from surrounding content. Recognizing these patterns does not extract metadata values as attributes associated with the recovered records.

Physical nodes carry a unique `node_id`, a type, a nonempty list of unique fields, visual block positions in `bid`, a child relationship, and complexity annotations. Node identifiers are lookup keys. A visual position is not a substitute for node identity, because two alternative schemas may occupy the same position.

Multi-tier headers produce names such as `Group - Column`. Remaining duplicate field names receive deterministic suffixes.

2.4.2 Exclusive-choice slots

A fixed physical node sequence is awkward when a record contains either table A or table B at the same logical position. Treating both as mandatory suggests that every record is incomplete; treating them as unrelated may introduce false restarts. TWIX 2 represents the logical position with a virtual `xor-table` or `xor-kv` node whose `includes` list refers to at least two physical alternatives of the matching type.

The XOR node is a grouping slot. It has no fields, produces no physical block, and has no extraction script. Observing an included alternative visits the shared slot during record traversal. Physical alternatives retain their own schemas and scripts, allowing each extractor to target one layout without flattening incompatible fields into a single table.

In the exported `id_23` template, virtual node 1 uses `includes=[2,3,4]` to associate three physical table schemas with one logical slot, as shown in figure 2.4. Figure 2.5 shows the corresponding document regions and the header and unit variations that distinguish the alternatives.

The source catalogue labels these table families HE [wide-flange H-sections], UB/UC [universal beams/columns], and MB/MC [metric beams/columns].

An independent optional section that can coexist with another section is not an XOR alternative. This distinction matters because the template guides both generated code and deterministic grouping.

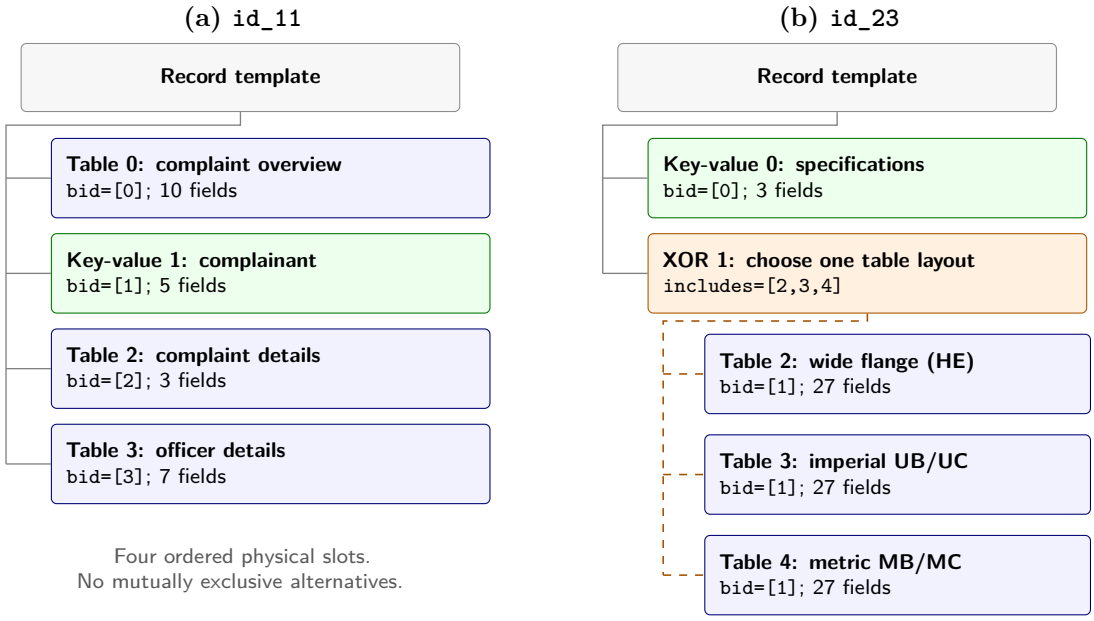
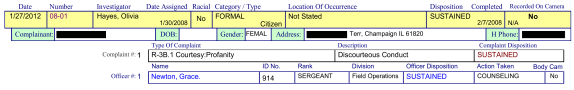
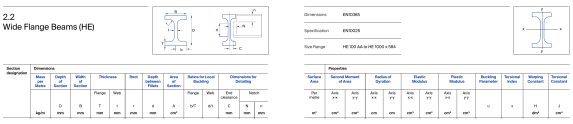


Figure 2.4: Exported template trees: four ordered physical blocks in `id_11`, and a key-value block followed by an XOR slot in `id_23`. Dashed branches link alternative table nodes 2, 3, and 4 at `bid=[1]`. Blue denotes tables, green key-value blocks, and orange the virtual XOR slot. Block descriptions are abbreviated.

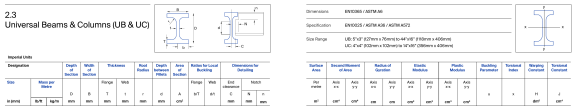
(a) `id_11`: one complete record
Source page 1; physical nodes 0, 1, 2, 3



(b) `id_23`: Table 2, HE
Source page 8; wide flange layout



(c) `id_23`: Table 3, UB/UC
Source page 13; imperial layout



(d) `id_23`: Table 4, MB/MC
Source page 20; metric layout

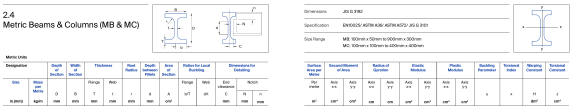


Figure 2.5: Document excerpts matching figure 2.4: (a) one complete `id_11` record, containing nodes 0 to 3; (b), (c), and (d) the tops of `id_23` pages 8, 13, and 20. These show the header and unit variations represented by alternative table nodes 2, 3, and 4.

2.5 Generation of physical block separation

The separator turns a page containing several structured regions into individual block occurrences that can be processed independently. During construction, the model receives the inferred template and sampled page text with geometry, and generates a program for locating the corresponding regions. When executed, this program takes words and coordinates from the current input and assigns detected regions to physical template nodes, retaining their source words and positions. For example, the overview table and personal-details form in one register record become separate blocks. Separating them allows each subsequent extractor to work with one known schema instead of interpreting the entire page.

Table detection requires a complete, unambiguous schema and layout match, considering leaf columns, group-header paths, units, order, and relative arrangement. Unsupported or ambiguous tables should remain unclassified rather than be assigned to the nearest node. This creates a precision-coverage tradeoff: rejection may prevent incorrect assignments while still leaving content unextracted.

Every block is page-local. A table spanning pages produces separate continuation blocks, and a block under page p contains only page-p words. Generation and execution are retried at most three times, with a 300-second separator execution timeout and basic JSON validation. Passing these checks establishes an executable artifact, not complete semantic correctness.

For each physical block, the runtime translates coordinates so that its top-left corner becomes $(0, 0)$. This reduces dependence on where a block happens to occur on the page. Translation alone does not provide scale invariance; generated logic must still derive distances and tolerances from local text size and spacing.

2.6 Block samples and extractors

2.6.1 Learning one physical node at a time

Block extractors convert the words inside an identified region into field-value outputs. Once separation succeeds, the framework collects sample occurrences for each physical template node and presents them, together with that node's schema, to the code generator. It generates a specialized routine that receives one block at a time, with word coordinates relative to the block origin. The routine can focus on assigning text to the expected fields and reconstructing rows, because region detection and record assembly are handled elsewhere. This is the central division of work: each generated extractor solves one recurring structural task. Figure 2.6 illustrates this decomposition for one `id_11` record.

Table extractors return rows keyed by the exact inferred fields. Key-value extractors return one dictionary or a list of dictionaries when a node repeats. Values remain textual content, and visually nested blocks are processed through their own scripts rather than turned into arbitrary nested cell objects. The extractor prompt specifies these contracts; see appendix A.4.

2.6.2 Interfaces and generation validation

A common interface allows the runtime to invoke independently generated extractors in the same way. Parsing and interface checks verify that each script has valid Python syntax and declares the required function before it is accepted into the bundle. These checks concern the program's form; extraction quality is assessed separately against reference outputs. Each physical-node script exposes exactly one synchronous top-level function with one positional argument:

```
def extract(block: dict) -> dict:
    ...
```

Before each attempt, the target script is removed so a stale file cannot count as success. Parsing and interface checks supply diagnostics to subsequent attempts, up to three in total. Failed attempts contribute to construction cost. Generated scripts use the standard library; the portable runtime additionally requires `pdfplumber`.

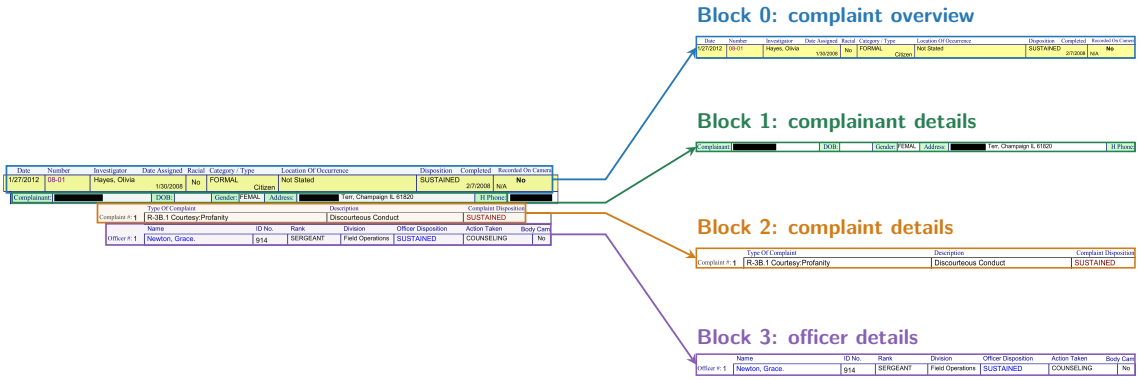


Figure 2.6: Decomposition of one `id_11` record from source page 1. Colored boundaries follow the four separator-assigned blocks, with a small padding for display. Matching arrows connect each region to its isolated crop: complaint overview (table 0), complainant details (key-value block 1), complaint details (table 2), and officer details (table 3). Each physical template node has its own extraction script.

2.7 Portable execution and deterministic record grouping

Execution applies the constructed pipeline to additional pages, while record grouping restores the relationships between separately extracted blocks. A table or form section may be extracted correctly on its own but still needs to be associated with the other sections describing the same case. The runtime therefore produces both individual physical-block outputs and logical records that collect related blocks. The exported bundle contains the runtime, template, separator, extraction scripts, record grouper, and requirements. Given a PDF from the same family, it obtains words and geometry, locates blocks, translates their coordinates to each block's origin, invokes the matching extractors, and assembles their outputs using fixed template traversal logic.

The grouping mechanism is taken directly from TWIX [1]: it traverses the template, using the principle of visiting each template node once per logical record to identify record boundaries. The grouper starts a new record on a completed template restart or a duplicate nonrepeatable node. Repeated nodes, multi-position nodes, and adjacent cross-page table continuations can remain in the current record. XOR alternatives visit their common logical slot. Importantly, grouping does not erase the separate physical blocks that support the record.

2.8 Training evaluation and bounded repair

Training evaluation checks how well the completed bundle recovers the evidence supplied during construction. Its inputs are the bundle's predictions and reference extractions for the training pages; its output is a diagnostic report distinguishing missing or extra records, missing or extra nodes, and incorrect values. This separates successful program execution from successful extraction. References are generated by a model from five-page OCR windows, and the bundle is evaluated on all training pages. An optional repair stage can use the report to revise generated separator and extractor scripts, while leaving the runtime, template, and record grouper fixed. Key instructions appear in appendix A.9.

Repair was disabled in all evaluations reported here. This isolates the proposed divide-and-conquer construction workflow from subsequent feedback-driven correction. Although useful in deployment, enabling repair would confound this comparison by adding an improvement mechanism beyond the pipeline under study.

2.9 Cost and scaling

Let N be the number of production pages, B the realized training evidence, K the number of distinct physical block schemas, and R the bounded repair budget. A useful accounting model is

$$\begin{aligned} C_{\text{total}}(N) = C_{\text{scan}} + C_{\text{construct}}(B, K, R) \\ + C_{\text{execute}}(N) + C_{\text{evaluate}}, \end{aligned} \tag{2.3}$$

Here, construction covers template inference, separator generation, extractor generation, and failed generation attempts. Reference generation for training and test pages, together with scoring, belongs to evaluation and is excluded from the construction costs used for break-even analysis. Repair was disabled, so evaluation feedback did not change the generated pipeline. Scanning and execution consume local computation even when they incur no API charge.

Once the extraction pipeline has been built and frozen, let N denote the number of additional pages it processes. The model cost during execution and the average model cost per page are

$$C_{\text{model,execute}}(N) = 0, \qquad \overline{C}_{\text{model}}(N) = \frac{C_{\text{model,construct}}}{N}. \tag{2.4}$$

The first formula states that running the generated extractor requires no further LLM calls, so processing additional pages incurs no additional model cost. Reading the PDFs and executing the generated Python code still consume computing resources.

The second formula spreads the initial model cost of constructing the pipeline across the pages processed. It therefore gives the average model cost per page.

Both formulas assume that the frozen pipeline remains suitable for the documents it processes. A new template variation that requires model-assisted reconstruction introduces an additional construction cost. Execution time also continues to grow with the input, since every page must still be read and processed.

2.10 The artifacts available to the user

The extracted output alone does not reveal whether the system has missed structured content in the source document. TWIX 2 preserves the inferred template, generated programs, block overlays, record overlays, split mappings, and evaluation reports. These artifacts let a user inspect whether an error began with schema inference, region detection, value recovery, or record assembly.

2.10.1 Coverage as a failure signal

Immediately after the separator passes validation, a module in the pipeline called document sonar applies it to up to 100 source pages from the structured-data start. For page p, let W_p be the set of OCR word identifiers and A_p the subset assigned to at least one detected block. Sonar measures

$$c_p = \frac{|A_p|}{|W_p|}, \qquad |W_p| > 0. \tag{2.5}$$

IDs are checked against the same scan's OCR input, preventing words from another page or invocation from inflating coverage. Pages without words have undefined coverage and are treated separately.

Writing $\bar{c}$ for the mean over pages with defined coverage and δ for a configured relative drop, a page is flagged when

$$c_p < \bar{c}(1 - \delta). \tag{2.6}$$

We measure coverage drops relative to the documents mean because the proportion of metadata to structured data varies across documents. A fixed threshold could flag many pages in documents with substantial metadata, even when extraction is working correctly, since metadata is intentionally excluded from the detected blocks.

When all defined coverage is zero, pages with OCR words are flagged directly. The resulting report preserves total words, covered words, excluded words, and flagged page numbers. Figure 2.7 illustrates a coverage drop caused by a table schema missing from the inferred template.

Coverage is a proxy for detection behavior, not an accuracy metric. A wrong extractor can cover every word, while a correct separator may intentionally exclude narrative or page metadata. Similar coverage across many wrong pages can also hide systematic failure. A decline should prompt inspection, and its meaning depends on the type of page and the

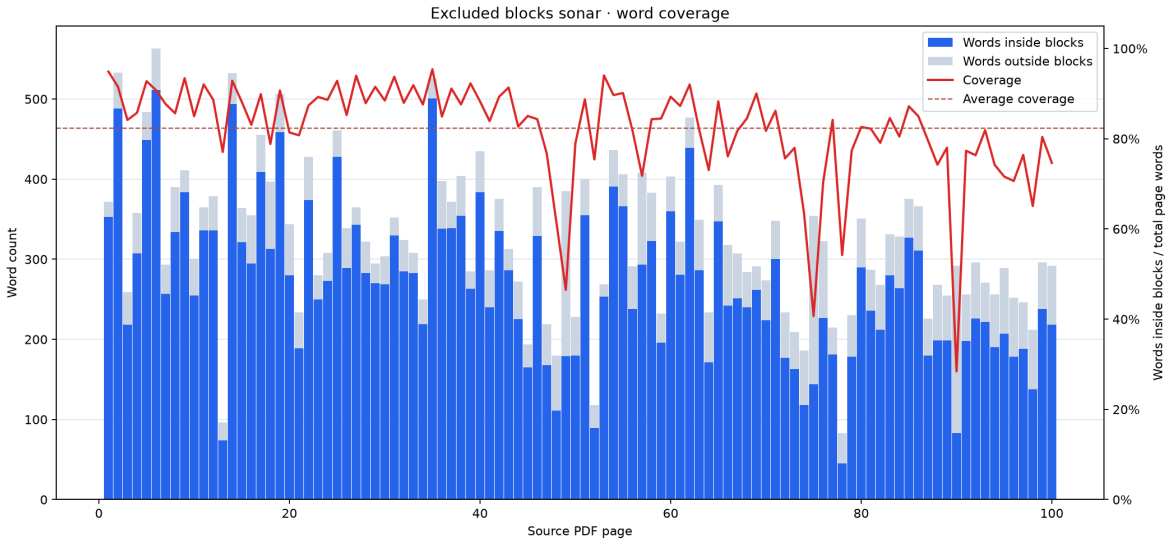


Figure 2.7: Sonar coverage for the first 100 source pages of `id_22`. On source page 90, only 83 of 292 words are assigned to blocks (28.4% coverage), compared with a scan mean of 82.3%. Inspection reveals a flange-dimensions table whose schema is absent from the inferred template, illustrating how sonar directs attention to potentially missed structured content.

intended structured content.

At present, sonar is intended only to guide inspection. A future extension could use coverage drops to identify candidate structures outside those represented in the training sample. The flagged pages could then provide additional evidence for adding new block schemas to the template and generating their extractors. In the evaluation presented here, sonar findings were not used to augment the training sample, extend the template, or revise the pipeline. This kept the training information budget aligned with the baseline to support a fair comparison.

Chapter 3

Experiment design

3.1 Document families and data splits

Documents are assigned identifiers `id_01` to `id_23` in the order used in figure 4.1. Group 1 contains `id_01` to `id_10`, Group 2 contains `id_11` to `id_20`, and Group 3 contains `id_21` to `id_23`. These identifiers are used consistently throughout the report. The dataset contains 23 PDFs spanning stable layouts, alternative table schemas, changing key-value arrangements, sparse rows, multi-tier headers, and cross-page continuations. Each pipeline is constructed from selected training pages and evaluated on disjoint pages of the same PDF. The total pages used from PDFs during the experiments are 117 for training and 440 for test. Training-page budgets match the baseline for every document: five pages for 22 PDFs and seven for `id_23`. Table 3.1 reports the page budgets; extraction results are presented in chapter 4.

The training budget was fixed at five pages for all documents except `id_23`, which contains numerous template variations. For this document, clustering indicated that five pages would not adequately represent the detected structural diversity. The budget was therefore set to seven pages, reflecting a practical deployment scenario in which the sample is expanded to include the identified structures. Both TWIX 2 and the baseline received the same seven-page budget, allowing the comparison to assess how effectively each approach uses the available evidence.

Structural groups and diagnostic purpose

We divide the PDFs by layout and field schema, not measured accuracy. Changes in values or row counts alone do not establish a new template. Figure 4.1 follows these groups.

Group 1: fixed page structure (10 PDFs). Consistent tables or form sections recur across pages. This group tests extraction under a stable schema, including wrapped text and hierarchical headers. Gains here indicate improved extraction logic without template variation.

Group 2: repetition and nesting (10 PDFs). Known blocks repeat in varying quantities, contain variable numbers of children, or continue across pages. This group tests block separation and parent-child association, probing the benefit of specialized extractors and deterministic record assembly over monolithic code.

Group 3: template variations (3 PDFs). Alternative schemas or additional block schemas appear in `id_21`, `id_22`, and `id_23`. This group tests whether clustering exposes variants within the training budget and whether expanded templates and generated code handle them.

These groups distinguish gains in extraction, record composition, and schema coverage.

Document	Train pages	Test pages
`id_01`	5	20
`id_02`	5	20
`id_03`	5	20
`id_04`	5	20
`id_05`	5	20
`id_06`	5	20
`id_07`	5	20
`id_08`	5	20
`id_09`	5	20
`id_10`	5	20
`id_11`	5	15
`id_12`	5	20
`id_13`	5	16
`id_14`	5	20
`id_15`	5	20
`id_16`	5	20
`id_17`	5	20
`id_18`	5	20
`id_19`	5	16
`id_20`	5	20
`id_21`	5	17
`id_22`	5	20
`id_23`	7	16
Total	117	440

Table 3.1: Training and test page budgets for the DeepSeek V4.1 Flash runs. Training-page counts match the monolithic baseline for every document.

3.2 Baseline and other comparison

The first experiment compares staged TWIX 2 construction using DeepSeek V4.1 Flash with monolithic code generation using GPT-6 Sol (section 4.1). Page budgets are given in table 3.1; baseline construction and evaluation are described in section 3.3.3. We also tested Qwen3.8 27B within TWIX 2 against DeepSeek V4.1 Flash.

The supporting comparison with original TWIX and its version with agent-generated tem-

plate and metadata uses a separate flat-pair metric and retains failed runs (section 4.2).

3.3 Evaluation

3.3.1 Reference generation and interpretation

Reference outputs are generated using AI-based extraction and are treated as operational evaluation targets, rather than error-free ground truth. Consequently, the reported scores measure agreement with these references. Reference omissions or transcription errors can reward shared mistakes or penalize correct predictions, so these scores do not independently establish absolute extraction correctness. Using LLM-generated references is consistent with the thesis objective of approaching the extraction quality achievable through direct LLM inference while retaining the scalability of reusable extraction programs.

3.3.2 Scoring and interpretation

Scoring first matches predicted records and blocks to their reference counterparts, then compares their contents. Hungarian assignment chooses one-to-one matches with the highest total similarity, first between records and then between their blocks. This avoids relying on output identifiers or record order. Within tables, exact row matches provide anchors, and banded Needleman-Wunsch alignment finds a high-scoring sequence of row matches and gaps within a restricted search range. It preserves row order while allowing missing or extra rows, preventing one omission from shifting every later comparison. Weak row pairs are rejected unless supported by an exact nonempty anchor.

The headline accuracy gives equal weight to recovering the expected blocks and recovering their contents. Typed-node F1 combines precision and recall for matched blocks of the correct type, penalizing missing, extra, or incorrectly typed blocks. Content similarity compares table headers and cell values, or key-value pairs, and weights each block by its number of cells or pairs. Combining these terms rewards both structural completeness and faithful value extraction. The evaluated representation contains no parent edges, so the scorer omits its parent-edge term and renormalizes the remaining weights:

$$\text{Accuracy} = 0.5F_{1,\text{typed node}} + 0.5S_{\text{content}}. \tag{3.1}$$

Missing and extra nodes contribute zero content, preventing a method from obtaining a high content score by retaining only easy outputs. Record matching and untyped node matching remain useful diagnostics but do not directly enter equation (3.1). XOR slots organize alternative schemas and are not scored separately; only their physical table or key-value blocks and extracted contents are evaluated.

The scorer normalizes compatibility characters, whitespace, case, dash and quote variants, surrounding formatting punctuation, numeric formatting, and some intra-token spacing. Nor-

malization improves tolerance to presentation differences but may also suppress distinctions relevant to a particular application. Sensitive identifiers and domain-specific formats should therefore receive an additional exact-value audit where appropriate.

3.3.3 Independent baseline construction and adaptation

Using the `pdf-extraction-baseline` skill, GPT-6 Sol inferred a template and generated a standalone monolithic extractor from the development pages. Generation instructions exposed neither TWIX methodology nor the scorer format; the only requirement introduced is producing a template; this template is necessary to enable later conversion and thus compatibility with the TWIX 2 scorer. The frozen extractor was evaluated on the next 20 available source pages after the development sample, or all remaining pages when fewer were available. The `pdf-extraction-eval-adapter` skill produced a deterministic adapter that preserved substantive fields and rows while converting predictions to the format required by the TWIX 2 scorer. The unchanged scorer evaluated these held-out predictions and, subsequently, predictions obtained by running each frozen extractor on its development pages. This separates independent program construction from adaptation and measures training and test accuracy consistently across systems. Complete instructions for both skills appear in appendices A.10 and A.11.

Chapter 4

Results and Discussion

The experiments use two distinct comparators. A monolithic code-generation baseline using GPT-6 Sol is compared with TWIX 2 in extraction accuracy and model cost of construction; both produce reusable programs. Direct LLM extraction, which invokes a model to recover data from each page, supplies the estimated per-page cost used to calculate when each program recovers its construction expense. It is a cost reference, not a separate direct-extraction accuracy benchmark.

4.1 Comparison with frontier models

This experiment tests this combined design against an independent GPT-6 Sol code-generation baseline on 23 PDFs. Both systems generate reusable programs and are evaluated with the same scorer defined in equation (3.1). Matched training-page counts control the amount of page evidence, while the comparison evaluates the complete construction approaches, including their different operational models. Table 4.1 summarizes their training and test accuracy.

Configuration	Train (%)	Test (%)	Gap (pp)	Test pages
TWIX 2, DeepSeek V4.1 Flash	97.99	98.72	−0.73	440
GPT-6 Sol, baseline	99.47	89.05	+10.42	440

Table 4.1: Recorded document-level macro accuracy on 23 PDFs. Gap is mean training minus test accuracy, computed before rounding.

The results support the expected improvement in generalization: TWIX 2 maintains high accuracy beyond its construction sample, while the baseline's strong sample fit does not transfer consistently to the test pages. The baseline achieves 99.47% mean training accuracy and 89.05% mean test accuracy, a 10.42 percentage-point gap. The corresponding TWIX 2 means are 97.99% and 98.72%, a gap of -0.73 percentage points.

The per-document results show that frontier-model code generation is already effective on

many PDFs, while larger failures reduce its overall reliability. The baseline's median test accuracy is 99.88%; nine documents score 100%, and 19 reach at least 90%. Figure 4.1 locates these differences within the structural groups defined in section 3.1, retaining all scored documents and failed-page predictions.

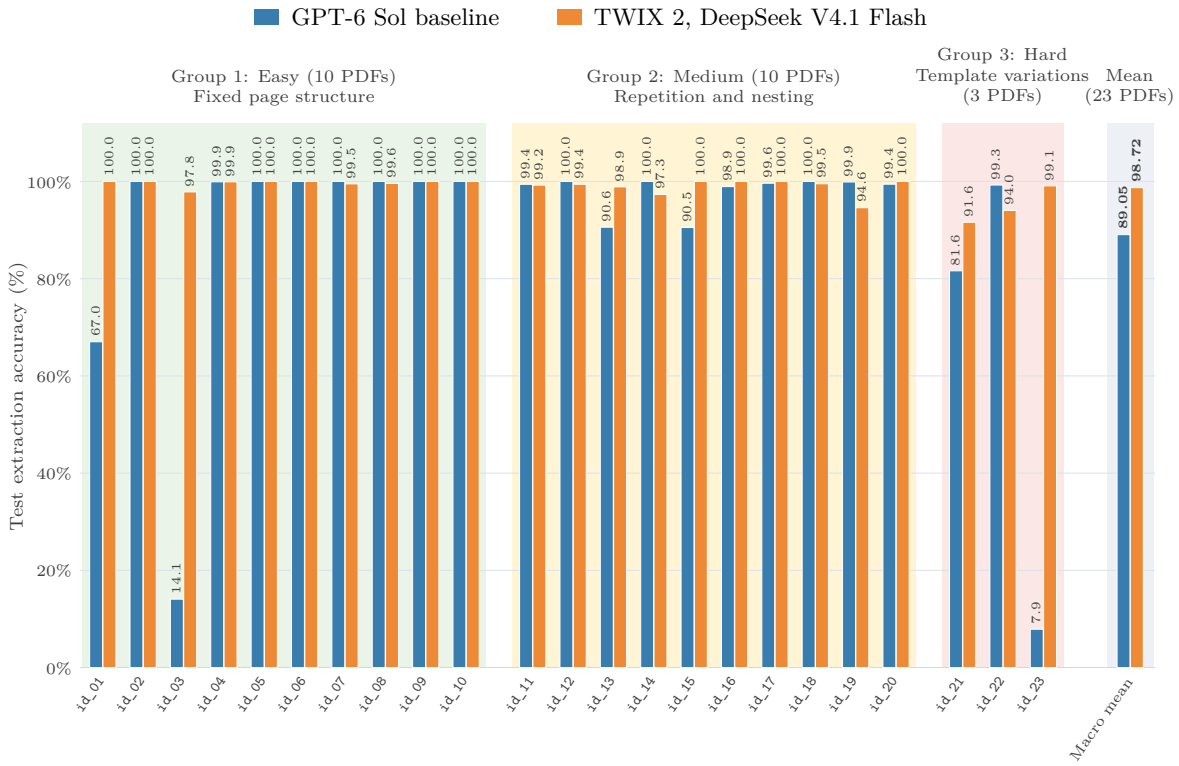


Figure 4.1: Baseline versus TWIX 2 test accuracy, ordered by document structure. Pastel green denotes a fixed page structure (10 PDFs), yellow denotes variable repetition or nesting within one template (10 PDFs), and red denotes template variations across pages (3 PDFs). Changes in row counts or values alone do not establish a new template. The mean gives equal weight to all 23 PDFs

4.1.1 Performance mismatch analysis

Inspection of the generated programs and targeted replays reveals why high baseline training accuracy does not always transfer to unseen pages. The main causes are restrictive assumptions about cell contents, dependencies on document context, and incomplete handling of repeated structures or schema variations. Detailed page-level evidence, replay outputs, and audit records are provided in appendix B.

Restrictive value assumptions. Assigning cells through their geometry and preserving printed text reduces dependence on assumptions about valid field contents. The inspected failures illustrate why this matters: in `id_01`, mandatory date parsing rejects a printed `Unknown` value; these assumptions can abort extraction even when the printed content is

readable. The corresponding TWIX 2 routines retain unusual values in their intended fields, illustrating the role of extraction contracts that prioritize faithful recovery of source content.

Fixed row-completeness assumptions. In `id_03`, the baseline requires every data row to contain nine populated numeric cells. An October observation with no numeric values violates this assumption, causing the extractor to raise an error and abort complete-document processing. The corresponding TWIX 2 routine preserves the printed row content without requiring a fixed number of populated cells, allowing it to process the incomplete observation.

Unseen layouts and repeated details. Representative sampling and explicit schema alternatives address a different source of failure: structures absent from the supplied examples or incompatible with a single fixed layout. In `id_23`, clustering exposes TWIX 2 to alternative schemas within the same seven-page budget, and XOR routing assigns their occurrences to specialized extractors. The contiguous baseline sample omits these alternatives, and its generated code requires a fixed number of column boundaries and restrictive section designations. Despite perfect training accuracy, the baseline reaches only 7.89% test accuracy, compared with 99.09% for TWIX 2. In `id_21`, multi-tier header recognition and preservation of repeated physical rows address the baseline's literal single-line header matching and restrictive flight-detail representation. Recorded accuracy increases from 81.61% to 91.60%.

Global constraints and remaining coverage gaps. In `id_22`, a document-wide uniqueness check rejects repeated part numbers and aborts the baseline. TWIX 2 retains both occurrences within their physical blocks and completes the diagnostic replay. However, its template still misses an additional table schema, and its recorded test accuracy is lower than the baseline's. Successful execution therefore does not establish complete coverage or higher accuracy on every document.

Summary of detected causes and improvements. Across the inspected documents, failures arise from strict value validation, missing preceding context, fixed layout assumptions, loss of repeated details, and incorrectly assuming that identifiers must be unique throughout the document. TWIX 2 addresses these through printed-text preservation, page-local extraction, representative sampling, explicit schema alternatives, and specialized block extractors followed by record assembly. Its runtime also contains extractor exceptions within individual blocks, limiting their propagation. These mechanisms help explain the stronger aggregate test performance and smaller train-test gap, while remaining errors show that completion alone is insufficient. The cases support the agentic approach as a promising route to reliable extraction at scale.

4.1.2 Cost and break-even analysis

The cost comparison asks whether the proposed construction workflow, using a less expensive operational model, recovers its generation expense earlier than monolithic frontier-model code generation. Both systems reuse code. We therefore calculate each pipeline's break-even relative to direct LLM extraction: the number of pages at which repeated per-page inference would cost as much as constructing the program.

If $F_{s,d}$ is the fixed model cost for system s and document d, and c_d is the estimated direct-extraction model cost per page, then

$$N^*_{s,d} = \frac{F_{s,d}}{c_d} = \frac{F_{s,d}\,P_d}{L_d}, \qquad c_d = \frac{L_d}{P_d}. \tag{4.1}$$

Here L_d is the estimated LLM-only extraction cost for the complete source PDF, and P_d is its number of extraction pages. A threshold of 20 pages means that the program recovers its construction cost after approximately 20 pages of the same workload. Lower values indicate earlier recovery. The figure shows continuous estimates; the smallest whole-page count is $\lceil N^*_{s,d} \rceil$.

Baseline fixed cost sums code-generation sessions, including repeated attempts. For TWIX 2, it sums template inference, separator generation, and extractor generation. Reference generation and scoring are excluded because repair was disabled and evaluation feedback did not revise the programs. Both systems use the same estimated direct-extraction cost per page for each document.

The measured costs support earlier recovery of construction expense for the TWIX 2 configuration on most documents. In figure 4.2, median break-even across all 23 PDFs is 5.51 pages for TWIX 2 and 18.00 pages for the baseline. TWIX 2 reaches break-even earlier on 19 documents; the baseline does so on four. The corresponding ranges are 0.75 to 163.22 pages and 6.47 to 767.87 pages. These results establish the cost behavior of the evaluated configurations, combining their model choices and construction workflows.

Figure 4.3 shows how the mean model cost per page changes with extraction volume across all 23 pipelines. The mean amortized construction cost crosses the mean direct-extraction cost at approximately 7.6 pages: this is the ratio of mean construction cost to mean direct-extraction cost per page, not the median of document-level break-even points. The curves assume reuse without reconstruction and exclude local execution costs.

4.2 Paired comparison with the original TWIX

The benefit of agentic construction could extend beyond improving the inferred template. Specialized generated routines can adapt physical extraction to a particular schema, while replacing a template alone leaves the original engine's row reconstruction and value-assignment rules unchanged.

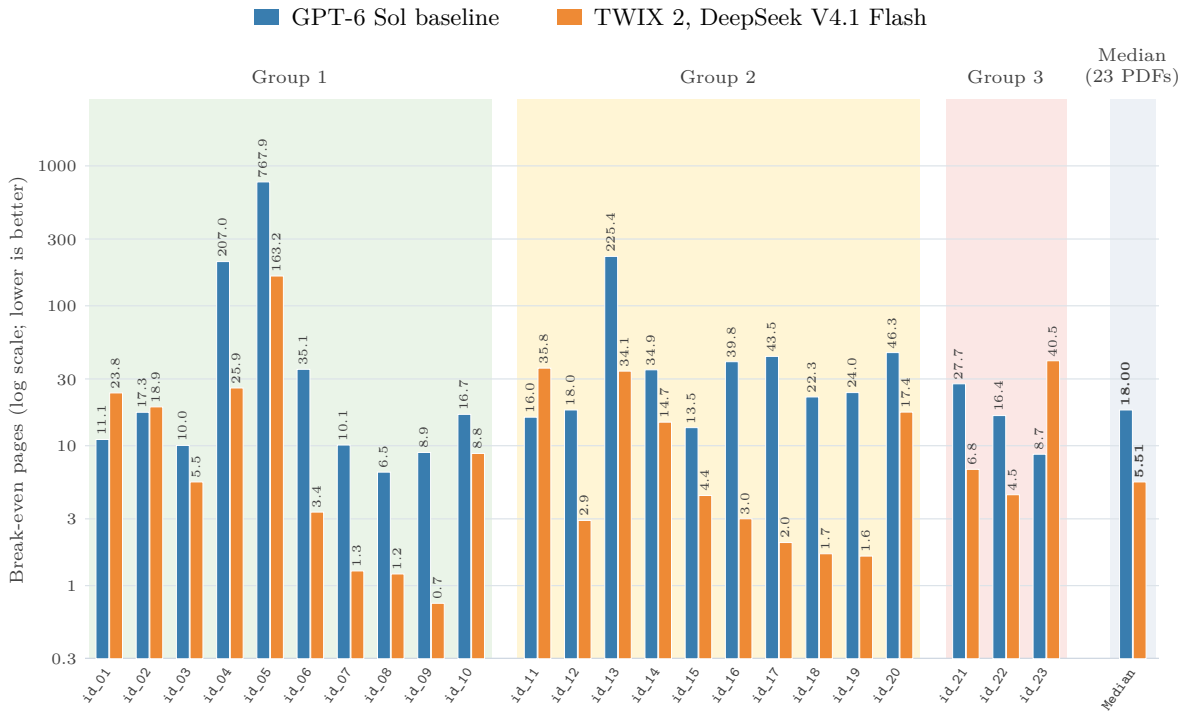


Figure 4.2: Break-even page counts for the baseline and TWIX 2, with the same document order, colors, and structural groups as figure 4.1. The separate group on the right shows the median for each system over all 23 documents. Lower is better; the vertical axis is logarithmic.

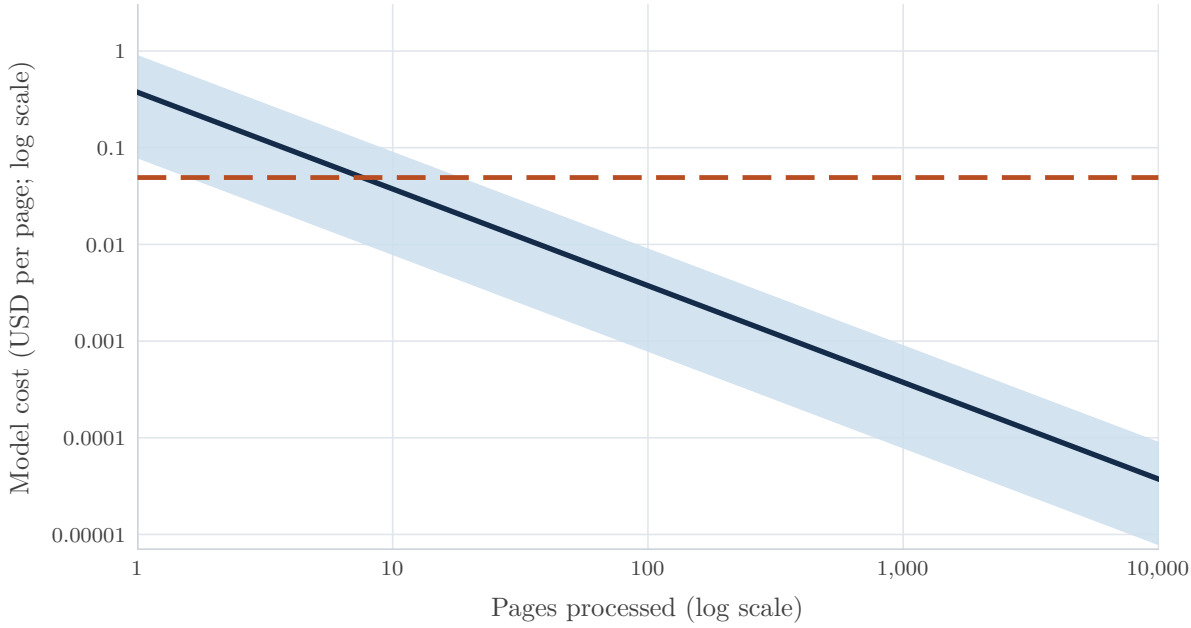


Figure 4.3: Mean model cost per page across 23 documents. Dark blue: amortized TWIX 2 generation cost; shading: 10th to 90th percentiles across documents. Dashed orange: direct LLM extraction (USD 0.0492/page). The mean curves cross at approximately 7.6 pages. Costs exclude local execution; see appendix C.

Configuration	Precision	Recall	F1
Original TWIX	27.40%	21.91%	22.88%
TWIX + agent template and metadata	35.72%	29.24%	30.59%
Full TWIX 2, DeepSeek V4.1 Flash	96.31%	96.65%	96.42%

Table 4.2: Document-level macro scores on 23 PDFs and 440 test pages. The agent package supplies a replacement template and metadata to the original extraction algorithm. Failed runs count as empty predictions.

This experiment therefore tests whether improved template inference is sufficient to obtain the full framework's performance.

We compare original TWIX, an improved version in which an agent generates both its template and metadata while retaining its extractor, and full TWIX 2 using DeepSeek V4.1 Flash. All three are scored on the same 23 PDFs, 440 test pages, and reference pairs. This supporting experiment retains its fixed pipeline configurations and common references, separately from the budget-matched main experiment.

For this version, an LLM-based agent infers both the template and metadata from source pages 1 to 5. These inputs are frozen before scoring and supplied to the unchanged original extractor, using the same phrase-extraction output. This equips the original system with recent model capabilities and tests whether improved structural inference can close the gap with full TWIX 2.

Following the original paper's flat key-value formulation [1], scoring uses exact matches between unique header/value pairs, with empty values represented as `missing`. Precision, recall, and F1 are macro-averaged across PDFs; failed runs produce empty predictions. This metric differs from the composite accuracy used in the main experiment.

The results indicate that improved template inference benefits the original system, while the full framework obtains a substantially larger gain. In table 4.2, agent-generated templates and metadata raise mean F1 from 22.88% to 30.59%, a gain of 7.71 percentage points, and all 23 runs of this version complete. Full TWIX 2 reaches 96.42%. The remaining gap motivates adapting downstream extraction as well as structural inference. Figure 4.4 shows the per-document results.

4.3 Comparison with Qwen 27B

We now examine whether the decomposed framework maintains high extraction quality when a smaller model generates the code. The motivation is that dividing extraction into narrower tasks may reduce the demands placed on the code-generation model.

Breaking extraction into smaller tasks is expected to make code generation accessible to smaller models. Each extractor handles a narrow block schema and local coordinates, while deterministic components manage execution and record assembly. This reduces the range of

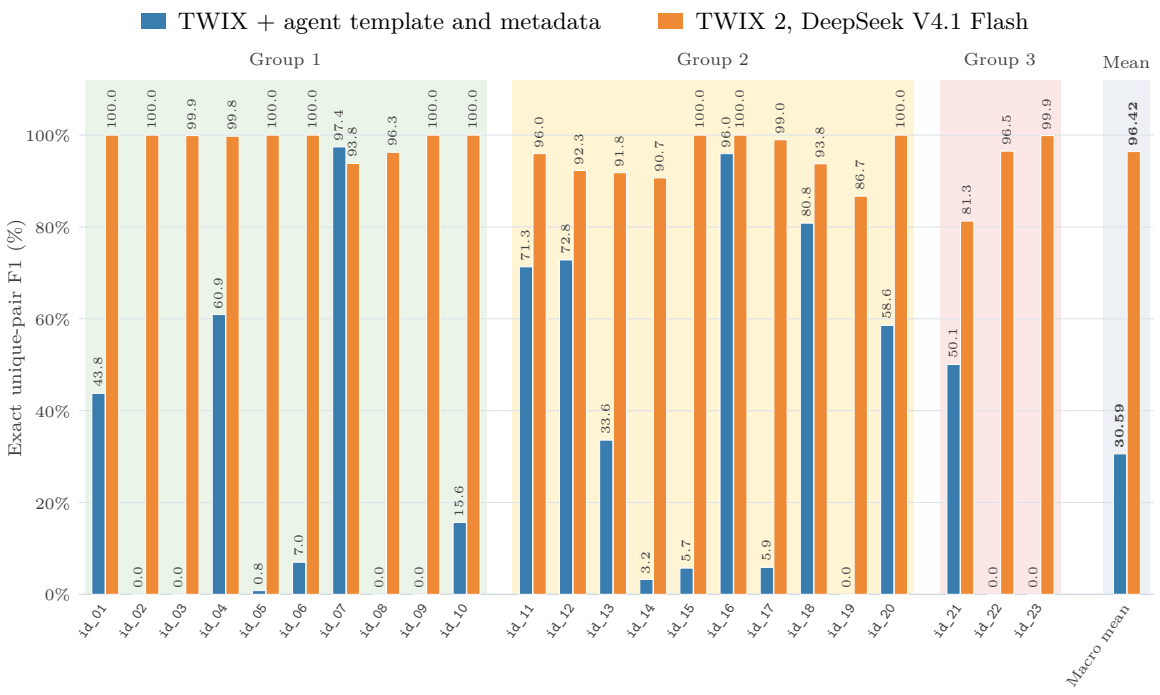


Figure 4.4: Per-document F1 for the original TWIX extractor with agent-generated template and metadata, compared with full TWIX 2, on the same 440 pages and reference pairs. Document order and structural-group backgrounds follow figure 4.1. The separate group on the right reports the macro mean across all 23 PDFs. Scores use exact unique key-value pairs.

layouts and decisions that an individual generated routine must handle. The experiment tests whether this division sustains high extraction quality when Qwen 27B replaces DeepSeek V4.1 Flash for code generation; DeepSeek continues to infer the template.

The results show that the smaller-model configuration retains accuracy close to the larger-model configuration. Across the same 23 documents, mean test accuracy is 97.31% with Qwen3.8 27B and 98.72% with DeepSeek V4.1 Flash, a gap of 1.40 percentage points. Qwen reaches at least 90% accuracy on 22 documents and at least 95% on 19. Figure 4.5 shows the per-document results and the frontier baseline's aggregate score. This supports high extraction quality with a smaller code-generation model within the decomposed framework.

The results motivate exploring fully local construction with models such as Qwen 27B. The completed program already executes locally; private construction would also require local template inference and code generation, plus any reference or repair models and associated artifact handling. These experiments used remotely hosted models. The effects of quantization, numerical precision, inference-engine implementation, context limits, and hardware on code quality, reliability, and speed remain to be evaluated.

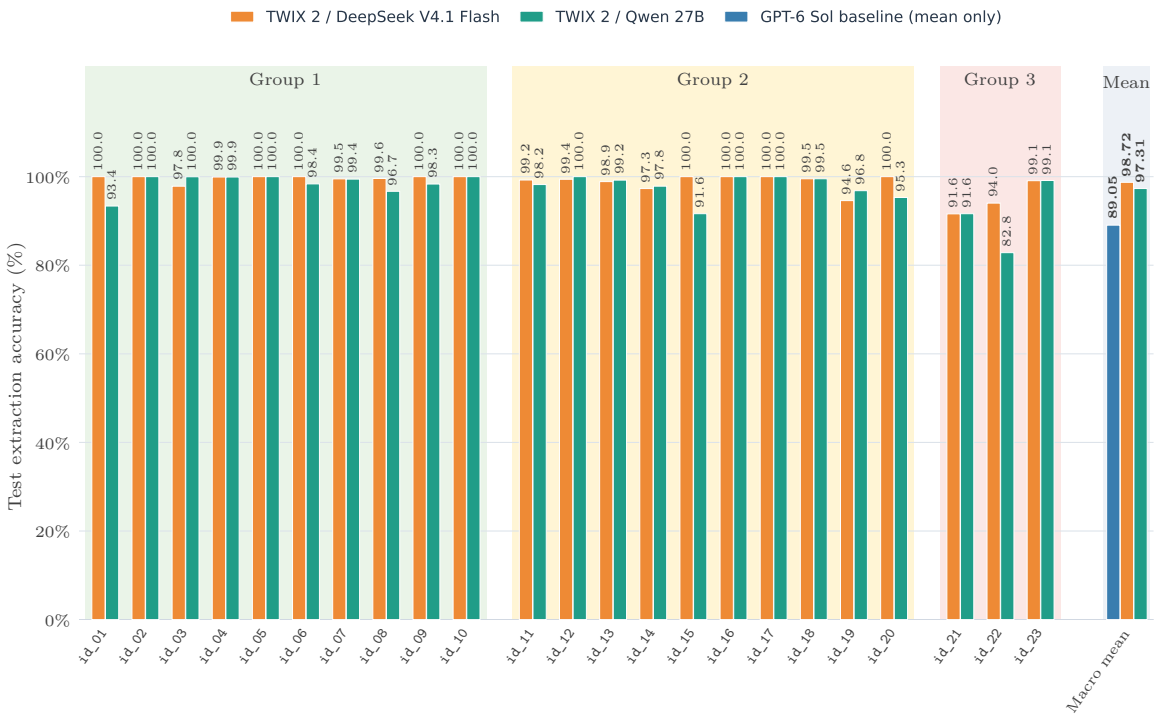


Figure 4.5: TWIX 2 test accuracy with DeepSeek V4.1 Flash and Qwen 27B across 23 documents. The rightmost group shows document-level macro means for both TWIX 2 configurations and the GPT-6 Sol baseline; baseline bars are included only in this aggregate group. Structural groups follow figure 4.1.

Chapter 5

Conclusion

The experiments support agentic pipeline construction as a promising direction for reliable extraction from templatized documents. Representative sampling exposes structural diversity, while explicit templates and specialized routines divide a complex extraction problem into more focused tasks. Together, these mechanisms help generated programs handle pages beyond their construction sample and provide intermediate evidence for inspecting failures.

The main experiment supports this advantage over monolithic frontier-model code generation: TWIX 2 achieves 98.72% mean test accuracy, compared with 89.05% for the GPT-6 Sol baseline. The baseline's 99.47% training accuracy shows that fitting the supplied pages does not ensure generalization to the rest of a document. Frontier models nevertheless achieve perfect or near-perfect agreement with the references on several documents with stable, repeated templates. Their limitation is consistency across the collection: unusual values, dependencies on preceding pages, repeated details, and later template variations can still cause failures or omissions. Six baseline programs fail on their complete PDFs, even when fallback evaluation produces usable scores on selected pages. The results demonstrate close agreement with AI-generated extraction targets at low amortized model cost.

The cost comparison also favors the proposed construction approach: median break-even relative to estimated direct LLM extraction is 5.51 pages for TWIX 2 and 18.00 pages for the baseline. Both systems reuse their generated code; the difference reflects the model expense of constructing their extractors under the evaluated workflows and model configurations. Reference generation and scoring are excluded because they did not contribute feedback to construction.

The supporting comparison with original TWIX reinforces the value of extending structural inference with specialized generated programs. On 23 PDFs and 440 pages, original TWIX achieves 22.88% F1. Equipping its unchanged extractor with an agent-generated template and metadata raises F1 to 30.59%, while full TWIX 2 reaches 96.42%. Improved template inference alone therefore leaves a substantial gap to the full framework. This comparison uses flat-pair F1, which is distinct from the main experiment's composite accuracy.

With Qwen3.8 27B generating the code, TWIX 2 reaches 97.31% mean test accuracy, close to the 98.72% obtained with DeepSeek V4.1 Flash. This supports decomposition as a promising way to make high-quality extraction accessible to smaller models. DeepSeek still supplies template inference in this configuration, and inference was remotely hosted; fully local quality, including the effects of quantization and inference implementation, remains to be evaluated.

A limitation of this thesis is the size of the evaluation dataset. Although assembled to represent the targeted extraction challenges, its limited size cannot capture the full diversity of document structures and variations.

5.1 Future work

Model capabilities changed substantially during the thesis. Early exploratory attempts with Opus 4.5 were expensive and produced code that failed to generalize beyond the examples. Later releases, particularly those explored in the final two months and represented by the frontier baseline, generated more effective programs at lower observed cost. These development observations are not a controlled comparison between model generations. They motivate combining newer models with TWIX 2: stronger code generation could complement its flexibility, while clustering, guided sampling, and inspectable outputs remain useful for exposing relevant structures and diagnosing failures.

One next step is to extend the existing optional post-construction repair into verification gates at individual stages. Checks of template coverage, block separation, extracted values, and record assembly could guide revisions before later stages proceed. Executable tests, schema constraints, geometry, and reliable reference values would provide verifiable rewards for code revision while keeping model weights fixed. The sonar module with coverage measurement could guide targeted sampling and template expansion, although coverage alone does not establish correctness. These extensions were outside the thesis's scope of decomposition, informative sampling, and reliable extraction contracts; repair was disabled in the benchmark, and the benefit of broader feedback remains unmeasured.

The framework is also naturally suited to human-in-the-loop workflows because it produces explicit templates, generated programs, block and record overlays, evaluation reports, and sonar module coverage diagnostics. These artifacts provide concrete points at which a reviewer could validate a schema, correct a block assignment, or supply feedback for pipeline revision. Given the need for reliable data extraction, this is a valuable direction to explore, particularly when building large databases from extensive document collections. Human effort spent reviewing and improving a reusable pipeline could be amortized across a large volume of subsequently processed pages, reducing the review cost per page while retaining targeted oversight.

To conclude, another possible path to explore is extracting metadata values and associating them with recovered records at scale, beyond the current recognition of framing-text patterns.

Relevant context may appear in headings, introductory text, or margins, with wording and placement that vary more than recurring tables and forms. Recovering and linking this context may therefore require more flexible detection and association mechanisms.

Bibliography

[1] Yiming Lin, Mawil Hasan, Rohan Kosalge, Alvin Cheung, and Aditya G. Parameswaran. *Visual Template Inference for Data Extraction from Documents*. arXiv:2501.06659, first submitted 2025; version 2, 8 June 2026. https://arxiv.org/abs/2501.06659v2.

[2] Carlos E. Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik Narasimhan. *SWE-bench: Can Language Models Resolve Real-World GitHub Issues?* International Conference on Learning Representations (ICLR), 2024. https://arxiv.org/abs/2310.06770.

[3] Wenqi Huang, Charley Lee, Leonard Tng, and Serena Ge. *DeepSWE: Measuring Frontier Coding Agents on Original, Long-Horizon Engineering Tasks*. arXiv:2607.07946, July 2026. https://arxiv.org/abs/2607.07946.

[4] Simran Arora, Brandon Yang, Sabri Eyuboglu, Avanika Narayan, Andrew Hojel, Immanuel Trummer, and Christopher Ré. Language Models Enable Simple Systems for Generating Structured Views of Heterogeneous Data Lakes. *Proceedings of the VLDB Endowment*, 17(2), pages 92 to 105, 2023. https://doi.org/10.14778/3626292.3626294.

[5] Shreya Shankar, Tristan Chambers, Tarak Shah, Aditya G. Parameswaran, and Eugene Wu. *DocETL: Agentic Query Rewriting and Evaluation for Complex Document Processing*. *Proceedings of the VLDB Endowment*, 18(9), pages 3035 to 3048, 2025. https://doi.org/10.14778/3746405.3746426.

[6] The pdfplumber contributors. *pdfplumber: PDF text, geometry, and table extraction library*. Project documentation, accessed 6 October 2026. https://github.com/jsvine/pdfplumber.

[7] Mantle. *Assets & Wealth Extraction Benchmark*. Mantle Research, updated 14 October 2026. https://research.withmantle.com/awe.

Appendix A

Full Prompts and Baseline Instructions

A.1 Structured-data start detection

```
You locate where the extractable, repeated structured-data template begins in a
PDF. You will receive one chronological batch of page OCR layouts. Each page is
identified by its physical PDF page number. Every OCR line includes y coordinates,
and every word includes its x span, so use both text and geometry as layout evidence.

Find the earliest page in this batch that belongs to the document's actual
structured records: tables, repeated key-value fields, forms, listings, or
other field-oriented content that the extraction pipeline should model.

Do NOT treat front matter as the template, even when it is visually tabular.
Front matter includes covers, title pages, copyright/legal pages, tables of
contents, indexes that point to later sections/pages, lists of figures or
tables, prefaces, and narrative introductions. A page counts as the beginning
when structured record content starts anywhere on it, even if a heading or
short introduction appears above that content.

If no page in this batch contains the actual structured records, use null.
Never guess a page outside the supplied batch.

IMPORTANT: the `confidence` value MUST be exactly one of the lowercase JSON
strings `"high"`, `"medium"`, or `"low"`. Never return `"High"`, `"LOW"`, a
number, a percentage, or any other confidence representation.

Return ONLY this JSON object, with no markdown:
{
  "start_page": <physical page number or null>,
  "confidence": "high",
  "reason": "one short explanation"
}

The example uses `"high"`; replace it with `"medium"` or `"low"` when that is
the appropriate confidence level. Do not return multiple alternatives.
```

A.2 Template inference

The source-field examples in this prompt use DOB [date of birth] and CH [channel].

```
You are a document structure inference agent. You will receive OCR layouts for the training
    pages of a PDF document collection that shares the same visual template. Each OCR line
    includes y coordinates and each word includes its x span. Use that geometry to
    reconstruct rows, columns, alignment, and whitespace. Your job is to analyze the
    document layout and return two things:

1. **The template**: the structural blueprint of repeating data blocks (tables and key-value
     sections).
2. **The metadata**: fixed text that frames the document but is not part of any data block.

You do NOT extract data values.

## How the pages were selected

The first three structured-data pages (or all available early pages) are the
**ordering anchors**. Use them to infer the normal top-to-bottom block order,
record boundaries, ‘bid‘ positions, and parent/child relationships. Other pages
were selected by deterministic table-header and key/value layout clusters and
may be far apart in the source PDF. Inspect every sampled page for additional
or alternate table and KV structures. A block that appears only on a sampled
page still belongs in the template; give a distinct table layout with different
fields its own node, even if it occupies the same visual slot as an earlier
table. Do not infer document order from the order of sampled source pages, and
do not let a continuation page redefine the block order established by the
first pages. Avoid adding a second node merely because one occurrence has a
partially detected or wrapped version of the same header.

## Core Concepts

- **Template**: An ordered list of physical table/kv nodes and, when needed, virtual XOR
    nodes describing mutually exclusive alternatives. It defines the structural blueprint
    used to generate all documents in the set.
- **Field**: A phrase that serves as a column header (in tables) or a key (in key-value
    pairs). Fields appear in consistent positions across records. They are structural labels,
     not data.
- **Value**: A data phrase that fills in a field. Values vary across records.
- **Metadata**: Text that appears as part of the document’s visual frame -- titles,
    organization names, page headers/footers, page numbers, section labels, disclaimers --
    but is NOT a field and NOT a value.
- **Record**: One complete instance of the template within a document. A document may
    contain multiple records, or one record may span multiple pages.
- **Data Block**: A contiguous visual region forming a single table or key-value section
    within a record. Each block has a block id (bid).

## Row Labels

Each visual row in a record is one of:
- **K (Key)**: A table header row -- most phrases are field names laid out horizontally.
- **V (Value)**: A table data row -- values visually aligned in columns under a K row.
- **KV (Key-Value)**: A row containing alternating field-value pairs side by side (a label
    followed by its value, horizontally in the same row).
- **M (Metadata)**: A header, footer, title, or free text -- not part of any data block.

---

## Part 1: Template Inference
```

```
### 1. Identify Records

Examine the pages to find where one record ends and another begins. Records generated from
    the same template will look visually similar -- same layout, same field labels, same
    arrangement of blocks. Look for repeating visual patterns across and within pages.

### 2. Identify Fields

Compare the recurring content across records:

- **Fields repeat identically** across records in consistent visual positions (e.g., "Date",
     "Name", "Invoice #" always appear in the same spot in every record).
- **Values change** across records (e.g., "05/01/2023", "Smith, John", "$1,400").
- Use both visual repetition and semantic judgment: field names tend to be descriptive
    labels, while values tend to be specific data.
- Pay careful attention to distinguish fields from values that happen to look like labels. A
     true field appears in the same position in every record.

### 2b. Collect All Structural Blocks Across Every Page

After identifying fields, scan **every page provided** for structured data blocks (tables
    and key-value sections) that you may not have seen yet in step 2:

- A block does **not** need to appear on every page or in every record to belong to the
    template. Some blocks are conditional or optional -- they appear only when the
    underlying data is present (e.g., a "Notes" kv section, an "Attachments" table that
    shows up in some records but not others).
- For each such block: ask "is this a structured data region (K/V/KV rows) or is it metadata
     (M)?". If it has field labels and data values in a table or key-value layout, it is a
    template node -- include it regardless of how rarely it appears.
- Only omit a block if you are confident it is pure metadata (title, header, footer,
    disclaimer) with no field-value structure.
- Position conditional nodes in the template in their natural top-to-bottom order relative
    to the other nodes.

### 3. Identify Block Types and Label Rows

For each visually distinct data region within a record:

- **Table block**: You see a header row of field names (column headers) with one or more
    data rows beneath it where values are vertically aligned under their respective headers.
     The header row is K, data rows are V.
  - **Multi-line headers do not change the block type.** Column headers that wrap across two
     or more lines within their cell are still a single K row -- the block is still a table.
     Classify it based on the presence of a header row followed by data rows, not on whether
     each header fits on one line.
  - **Wrapped cell values do not change the block type.** If a data value spans multiple
    lines within its cell (e.g., a long name broken across lines), the row is still a V row.
     The block is still a table.
  - **Sparse or checkbox-style columns do not change the block type.** Columns with single-
    letter or abbreviated headers (e.g., "M", "Tu", "W", "Th") whose cells contain markers
    like "X" or are blank are table columns, not key-value pairs. Do not confuse "header:
    marker" with a KV layout -- if these columns share a header row with other table columns
     and their values appear in aligned rows beneath them, the whole region is a table.
  - **Two-tier (group) headers.** Some tables have a header row of group labels spanning
    several sub-columns, with a second header row naming the sub-columns (e.g. "Balance", "
    Exports", "Imports" each spanning "Total", "Goods", "Services"). Both rows together form
     one K band. To keep field names unique, prefix each sub-column with its group label
    joined by " - " (e.g. "Balance - Total", "Balance - Goods", "Exports - Total"). Assign
    group labels deterministically from the header row above, left-to-right: each sub-column
     takes the group label whose horizontal span covers it. Sub-columns with no group label
    above them (e.g. a leading "Period" column) keep their own name unchanged.
```

```
- **Key-Value block**: You see field-value pairs arranged horizontally -- a label
    immediately followed by its value in the same row, often in a form-like layout. These
    rows are KV. A KV block has **no** shared header row above it; each row is self-
    contained with its own label and value.
- **Metadata**: Titles, section headers, page headers/footers, free text that is not part of
     a table or key-value structure. These are M.

Structural constraints:
- A K row must have at least one V row visually aligned beneath it.
- A V row must have at least one K row visually aligned above it.
- KV rows contain both fields and values in the same row.
- **The decisive test**: if a block has a header row (even a multi-line one) followed by
    repeating data rows whose values align under that header, it is a table -- regardless of
     how the headers or values are rendered.

### 4. Assemble the Template

From the identified blocks within one record:

1. Each **table header (K row)** becomes a node with `"type": "table"`. Its fields are the
    column headers.
2. Consecutive **KV rows** merge into one node with `"type": "kv"`. Its fields are the keys
    (not the values).
3. V rows confirm the table structure but do not create new nodes.
4. M rows are metadata and do not create nodes.

Assign each node a sequential `node_id` starting from 0, in the order established
by the first structured-data pages. Place later conditional or alternate nodes
next to the corresponding block position in that order.

When one record has exactly one of several mutually exclusive table layouts at
the same logical position, add a virtual `"xor-table"` node immediately before
the physical `"table"` alternatives. Use `"includes"` to list their node IDs.
Likewise, use `"xor-kv"` for mutually exclusive KV layouts. Keep every physical
alternative as its own ordinary table/kv node with its own fields, bid, child,
node_id, and complexity. The XOR node has only `type`, `node_id`, and `includes`:
it has no fields, bid, child, or complexity and never represents a physical
block. Give alternatives at the same logical position the same `bid` index.
Use XOR only when the alternatives substitute for one another within a record,
not for independent optional blocks that may coexist. For example, if records
have a shared KV section followed by either Dimensions A or Dimensions B,
model the shared KV normally and wrap the two dimensions tables in one XOR node.

Assign each node a `bid` array containing the block indices it generates within one record.
    Most nodes produce a single block (e.g., `"bid": [0]`). If a node generates multiple
    blocks within one record (as in nested/repeated sub-tables), list all block indices (e.g
    ., `"bid": [2, 4]`).

Determine nesting by visual overlap:
- If two blocks are **visually separate** (one ends entirely before the other begins, no
    interleaving), their nodes are **siblings**. Set `"child": -1`.
- If block A **contains** block B (rows of A appear both above and below B, or B is visually
     nested within A's boundaries), then B's node is a child of A. Set A's `"child"` to the
    `node_id` of its first child node.
- `"child": -1` means the node has no children (it is a leaf).

**Deduplication**: if two visually identical blocks in the same record have the same type
    and fields (e.g., a sub-table that repeats), they come from the same template node --
    list both block indices in that node's `bid` array.

### 5. Assess Block Complexity
```

```
For every node, add a `complexity` object that characterises how hard the block is to
    extract. The fields differ by block type:

**For `kv` nodes**, set:
- `repeated` (boolean): `true` if the kv block appears **more than once** within a single
    record (i.e., the same key-value layout repeats as a group, like multiple address
    entries). `false` if it appears exactly once per record.

**For `table` nodes**, set:
- `same_row_structure` (boolean): `true` if every data row in the table follows the same
    visual structure -- each row has one value per column, aligned consistently. `false` if
    rows can differ structurally (e.g., some rows span multiple columns, some rows are blank
     separators, some have extra sub-labels).
- `nested` (boolean): `true` if the cells of this table contain structured content beyond a
    plain value -- for example, each row cell itself contains a small table, a list, or key-
    value pairs. `false` if each cell contains a single plain value.

---

## Part 2: Metadata Identification

Scan the pages for text that is NOT a field and NOT a value. Metadata is the fixed visual
    frame of the document.

### What to include as metadata:
- Organization names and logo text (e.g., "Olanyville Police Department", "SECRETARIA DA
    ADMINISTRACAO")
- Document titles (e.g., "Complaints By Date", "INVOICE", "RELATORIO DE COMPRAS")
- Page numbers and their surrounding text (e.g., "Page 1 of 20", "Pagina 1 de 2707")
- Repeated header/footer phrases on every page (e.g., "Confidential", "Draft")
- Report parameters or filter descriptions (e.g., "Report Criteria: Complaints Occurred
    Between 1/1/2023 AND 12/31/2023")
- Section labels that introduce a data block but are not themselves a field (e.g., "
    Complaint #: 1", "Officer #: 2")
- Disclaimers, footnotes, watermarks rendered as text

### What NOT to include as metadata:
- Table column headers -- these are fields (they go in the template)
- Key-value keys -- these are fields (they go in the template)
- Any data values

### Metadata type

Classify each metadata item as one of:

- **`document_level`**: appears once (or a fixed number of times) for the entire document
    regardless of how many records it contains. Examples: the organisation name, the report
    title, a disclaimer, a date range that describes the whole report.
- **`record_level`**: tied to a single record and repeats (once per record) as records
    change. Examples: a complaint number that increments with each record, an officer index,
     a per-record date, a page number that resets per record.

### Metadata pattern

Do **not** store the raw text value. Instead, write a Python `re`-compatible regex pattern
    that matches any valid instance of that metadata item across all records and pages.

Rules for building the pattern:
- Keep static text exactly as it appears (escape special regex characters where needed).
- Replace a dynamic numeric part with `\d+`.
- Replace a dynamic date with the appropriate pattern, e.g. `\d{1,2}/\d{1,2}/\d{4}` or `\d
    {4}-\d{2}-\d{2}`.
```

```
- Replace a dynamic alphabetic part with `[A-Za-z]+`.
- Replace a dynamic alphanumeric part with `[A-Za-z0-9]+`.
- Replace a free-form variable-length value with `.+`.
- Anchor patterns tightly -- prefer specific character classes over `.+` when the format is
    predictable.

---

## Output Format

Return ONLY a single JSON object with two keys: `"template"` and `"metadata"`.

```json
{
    "template": [
        {
            "type": "table",
            "fields": ["field1", "field2", "field3"],
            "bid": [0],
            "child": -1,
            "node_id": 0,
            "complexity": {
                "same_row_structure": true,
                "nested": false
            }
        },
        {
            "type": "kv",
            "fields": ["key1", "key2", "key3"],
            "bid": [1],
            "child": -1,
            "node_id": 1,
            "complexity": {
                "repeated": false
            }
        }
    ],
    "metadata": [
        {"pattern": "metadata phrase 1", "type": "document_level"},
        {"pattern": "Page \d+ of \d+",  "type": "document_level"},
        {"pattern": "Record #: \d+",      "type": "record_level"}
    ]
}
```

Template Node Fields

- `type`: `"table"` or `"kv"` for physical blocks; `"xor-table"` or `"xor-kv"` for a virtual
     alternative slot.
- `fields`: Array of field name strings exactly as they appear in the document. For tables,
    these are the column headers. For kv blocks, these are the keys (NOT the values).
- `bid`: Array of integer block indices this node generates within a single record.
    Sequential starting from 0, in top-to-bottom order.
- `child`: The `node_id` of this node's first child, or `-1` if no children (leaf node).
- `node_id`: Sequential integer starting from 0, in top-to-bottom document order.
- `complexity`: An object describing the structural complexity of the block. Fields depend
    on `type`:
  - For `kv` nodes: `{ "repeated": <bool> }` -- `true` if the block appears more than once
    within a single record.
  - For `table` nodes: `{ "same_row_structure": <bool>, "nested": <bool> }` -- `
    same_row_structure` is `true` when every data row has the same layout; `nested` is `true
    ` when cells contain structured content (sub-tables, lists, kv pairs) rather than plain
```

````
    values.
- `includes`: Required only for XOR nodes. List at least two distinct `node_id` values of
    physical nodes with the matching type. A physical node may belong to only one XOR node.
    The XOR slot is considered visited when any one included node is visited.

### Metadata Array

Each element is an object with two keys:
- ``"pattern"`: a Python `re`-compatible regex string that matches any valid instance of this
     metadata item. Use exact text for static content and character-class patterns for
    dynamic parts (see pattern rules in Part 2).
- ``"type"`: either ``"document_level"` (appears once per document) or ``"record_level"` (
    repeats once per record).

Additional rules:
- No two entries should match the same text -- deduplicate by meaning, not by raw value.
- If no metadata is found, use an empty array `[]`.
- Escape regex special characters in static text (e.g. `\.` for a literal dot, `\(` for a
    literal parenthesis).

---

## Examples

### Flat Example (Police Records)

```json
{
    "template": [
        {
            "type": "table",
            "fields": ["Date", "Number", "Investigator", "Date Assigned", "Racial", "
    Category / Type", "Location Of Occurrence", "Disposition", "Completed", "Recorded On
    Camera"],
            "bid": [0],
            "child": -1,
            "node_id": 0,
            "complexity": { "same_row_structure": true, "nested": false }
        },
        {
            "type": "kv",
            "fields": ["Complainant", "DOB", "Gender", "Address", "H Phone"],
            "bid": [1],
            "child": -1,
            "node_id": 1,
            "complexity": { "repeated": false }
        },
        {
            "type": "table",
            "fields": ["Type Of Complaint", "Description", "Complaint Disposition"],
            "bid": [2],
            "child": -1,
            "node_id": 2,
            "complexity": { "same_row_structure": true, "nested": false }
        },
        {
            "type": "table",
            "fields": ["Name", "ID No.", "Rank", "Division", "Officer Disposition", "Action
    Taken", "Body Cam"],
            "bid": [3],
            "child": -1,
            "node_id": 3,
````
````

```
            "complexity": { "same_row_structure": true, "nested": false }
        }
    ],
    "metadata": [
        {"pattern": "Olanyville Police Department",
         "type": "document_level"},
        {"pattern": "Complaints By Date",
         "type": "document_level"},
        {"pattern": "Report Criteria: Complaints Occurred Between .+ AND .+",
         "type": "document_level"},
        {"pattern": "Complaint #: \d+",
        "type": "record_level"},
        {"pattern": "Officer #: \d+",
        "type": "record_level"}
    ]
}
```

Nested Example (Invoice)

```json
{
    "template": [
        {
            "type": "kv",
            "fields": ["Invoice #", "Invoice Month", "Invoice Period", "Account", "Product"],

            "bid": [0],
            "child": -1,
            "node_id": 0,
            "complexity": { "repeated": false }
        },
        {
            "type": "table",
            "fields": ["Line", "Start Date", "End Date", "Description", "Start/End Time"],
            "bid": [1],
            "child": 2,
            "node_id": 1,
            "complexity": { "same_row_structure": false, "nested": true }
        },
        {
            "type": "table",
            "fields": ["#", "CH", "Day", "Air Date"],
            "bid": [2, 4],
            "child": -1,
            "node_id": 2,
            "complexity": { "same_row_structure": true, "nested": false }
        },
        {
            "type": "table",
            "fields": ["Start/End Date", "Length", "Ad ID"],
            "bid": [3, 5],
            "child": -1,
            "node_id": 3,
            "complexity": { "same_row_structure": true, "nested": false }
        }
    ],
    "metadata": [
        {"pattern": "INVOICE",            "type": "document_level"},
        {"pattern": "Page \d+ of \d+",  "type": "document_level"}
    ]
}
```

```
```

Alternative Table Example

Each record has the same KV section and exactly one dimensions layout. The
physical table nodes share block position 1; the virtual XOR node is a grouping
slot and has no extracted block.

```json
{
    "template": [
        {"type": "kv", "fields": ["Name"], "bid": [0], "child": -1, "node_id": 0, "
    complexity": {"repeated": false}},
        {"type": "xor-table", "node_id": 1, "includes": [2, 3]},
        {"type": "table", "fields": ["Width", "Height"], "bid": [1], "child": -1, "node_id":
     2, "complexity": {"same_row_structure": true, "nested": false}},
        {"type": "table", "fields": ["Diameter", "Depth"], "bid": [1], "child": -1, "node_id
    ": 3, "complexity": {"same_row_structure": true, "nested": false}}
    ],
    "metadata": []
}
```

Two-Tier Header Example (Trade Statistics)

A table whose header spans two rows -- a group row ("Balance", "Exports", "Imports") above a
     sub-column row ("Total", "Goods", "Services" repeated under each group), with a plain "
    Period" column on the left:

```json
{
    "template": [
        {
            "type": "table",
            "fields": ["Period", "Balance - Total", "Balance - Goods", "Balance - Services",
     "Exports - Total", "Exports - Goods", "Exports - Services", "Imports - Total", "Imports
     - Goods", "Imports - Services"],
            "bid": [0],
            "child": -1,
            "node_id": 0,
            "complexity": { "same_row_structure": true, "nested": false }
        }
    ],
    "metadata": [
        {"pattern": "U\.S\. International Trade in Goods and Services", "type": "
    document_level"},
        {"pattern": "Page \d+ of \d+",                                    "type": "
    document_level"}
    ]
}
```

Note how every field name is unique: each sub-column is prefixed with the group label
    directly above it, joined by " - ", assigned left-to-right.

Rules

- Infer the template and identify metadata ONLY. Do not extract data values.
- Multi-line Field Handling: When a field name spans multiple lines in a header row (e.g., "
    Spots/" on one line and "week" on the next), preserve the line break in the field name

```
    by inserting a space before the continuation. Record the field as "Spots/ week" rather
    than concatenating it into "Spots/week". This maintains visibility of the original
    layout and prevents ambiguity in field identification.
- Base your inference on visual patterns across **all** provided pages, not just those that
    repeat on every page.
- **Include every structured data block (table or kv) seen on any page**, even if it appears
     in only one record or on only one page. A block is part of the template if it has field
     labels and data values in a table or key-value layout -- frequency of appearance does
    not matter.
- The only reason to exclude a structured-looking region is if it is clearly metadata (a
    title, header, footer, page number, disclaimer, or section label with no data values
    beneath it).
- Every physical table/kv node must have at least one field; XOR nodes have no fields.
- **Field names within a node MUST be unique.** Never output the same field name twice in
    one `fields` array. When a table has a group-header row spanning sub-columns (e.g. "
    Balance" / "Exports" / "Imports" each spanning "Total", "Goods", "Services"), prefix
    each sub-column with its group label joined by " - " (e.g. "Balance - Total"), assigning
     group labels deterministically from the header row above, left-to-right.
- Only include field names in `fields` arrays -- never include values.
- Template node order must reflect top-to-bottom document order.
- A phrase is either a field (goes in the template), a value (excluded entirely), or
    metadata (goes in the metadata array). No phrase should appear in both the template and
    the metadata.
- Every metadata entry must be an object with both `"pattern"` and `"type"` keys. Never put
    raw text or plain strings in the metadata array.
- Report all text exactly as it appears in the document (preserve original casing, spelling,
     and punctuation).
- If you cannot confidently determine nesting, default to `"child": -1` for all nodes.
- If a page is mostly a continuation of a record (e.g., more value rows of the same table),
    do not create a new node -- it belongs to the same node identified from the header.
- If fewer than 3 pages are provided, work with what is available.
- Every physical table/kv node must include a `complexity` object with the correct fields
    for its type. XOR nodes do not have `complexity`.
- Return ONLY the JSON object. No additional text, explanation, or markdown formatting.
```

A.3 Block separation

```
Create a Python script called `block_separation.py` that detects and extracts all blocks
    defined in the document template from every page of the OCR data.

Context:
- A sample OCR CSV file is at: <BUILD_OCR_CSV_PATH>
  Columns: id, page, x0, top, x1, bottom, text
  `id` is a globally unique integer assigned to each extracted word across the whole
    document.
- The document template defines the following blocks:
[
  {
    "type": "kv",
    "node_id": 0,
    "bid": [
      0
    ],
    "child": -1,
    "fields": [
      "Record ID",
      "Date"
    ],
```

```
    "complexity": {
      "repeated": false
    }
  },
  {
    "type": "xor-table",
    "node_id": 1,
    "includes": [
      2,
      3
    ]
  },
  {
    "type": "table",
    "node_id": 2,
    "bid": [
      1
    ],
    "child": -1,
    "fields": [
      "Item",
      "Length"
    ],
    "complexity": {
      "same_row_structure": true,
      "nested": false
    }
  },
  {
    "type": "table",
    "node_id": 3,
    "bid": [
      1
    ],
    "child": -1,
    "fields": [
      "Item",
      "Width",
      "Height"
    ],
    "complexity": {
      "same_row_structure": true,
      "nested": false
    }
  }
]
- The template's metadata patterns (Python `re`-compatible regexes) are:
[
  {
    "pattern": "Page \\d+ of \\d+",
    "type": "document_level"
  }
]

Each template node has:
  "node_id"  : integer block identifier
  "type"     : "table" or "kv"
  "fields"   : list of field/column names that appear as visible labels or column headers

Each metadata entry has:
  "pattern"  : a Python `re` regex matching one metadata phrase
  "type"     : "document_level" (page frame: titles, headers, footers, page numbers)
```

```
               or "record_level" (repeats once per record)

The script must define a main function with this exact signature:

  def main(inputpath: str, out_path: str) -> None

  - `inputpath`: path to the OCR CSV file to process.
  - `out_path`: path to the folder where `block_separation.json` must be written.

---------------------------------------------------------------
ALGORITHM
---------------------------------------------------------------

STEP 1 -- Load and group words into rows (page-local)
  Read the CSV. For each page, group words into rows by y-overlap:
  a word joins an existing row if its vertical span overlaps every word already in that
  row (y_i1 <= y_j2 AND y_i2 >= y_j1). Otherwise it starts a new row.
  Sort rows top-to-bottom by their minimum `top` coordinate.

STEP 2 -- Strip structural noise (headers, footers, page numbers)
  Before block detection, remove rows that are page-level noise. A row is noise ONLY if
  it satisfies BOTH of the following:
    (a) Its position is either:
        - near the page TOP: within the top 5% of the page's word coordinate range, OR
        - near the page BOTTOM: within the bottom 8% of the page's word coordinate range.
    (b) The row's text (words joined left-to-right with single spaces) matches one of
        the "document_level" metadata patterns above (use `re.search`).
  A row that does not match any document-level metadata pattern must be KEPT, no matter
  how few words it contains -- the last data row of a table often sits at the very
  bottom of a page and must never be stripped.
  Do NOT use the page's printed number value for this check -- use position plus the
  metadata patterns only. Hard-code the document-level patterns above into the script
  as a list of regex strings.

STEP 3 -- Detect and strictly validate block headers
  A block's header row is the row (or contiguous group of rows) where the block's
  field labels appear. Detect it as follows:

  For "table" nodes:
    Require a COMPLETE, UNAMBIGUOUS match of BOTH the fields and the table layout.
    A similar-looking table or a table containing all expected fields plus extra
    columns is NOT a match. Prefer leaving an incompatible table unclassified over
    assigning it to the wrong node_id.

    During generation, inspect the sample OCR to establish each node's structural
    signature: ordered leaf columns, full parent/group-header paths, units, number
    of header tiers, parent spans, and relative column arrangement. The template
    JSON supplies field identities; the sample OCR supplies their physical layout.
    Bind this signature to that node_id. Do not infer a replacement signature from
    an arbitrary runtime candidate, broaden the signature to accept incompatible
    sample tables, invent node IDs, or modify the template. If no sample provides
    an unambiguous complete reference, report that node as unsupported rather than
    creating a permissive fallback.

    At runtime, isolate a candidate's header band using local geometry and the
    transition to body rows. Reconstruct header cells within their own column spans
    and associate each leaf with its enclosing group headers. Require ALL of:
      - Every expected field maps to exactly one distinct leaf column, and every
        observed leaf column maps to exactly one expected field. No missing, extra,
        duplicated, merged, or split columns are allowed.
      - Full field labels, units, column order, header hierarchy, and group spans
        match the reference. A repeated label such as "Axis x-x" must be matched
```

```
        under its own parent (e.g. Elastic Modulus versus Plastic Modulus).
      - Relative column positions/widths and header-tier arrangement match the
        reference after translation and uniform scaling, with only small tolerances
        for OCR coordinate jitter and harmless wrapping inside the same header cell.
        Derive tolerances from local text size and spacing; do not permit arbitrary
        column movement, added tiers, or changed parent/child structure.

    Join wrapped words only within the same header cell in reading order. Never
    match a field using an unordered union of tokens from different columns, units,
    body rows, or an arbitrarily enlarged row range. Shared words elsewhere in the
    table cannot supply a missing field. Do not use majority/percentage matching,
    fuzzy similarity, or a nearest-template fallback. If multiple nodes match and
    layout cannot distinguish them, reject the candidate as ambiguous.

    Normalise whitespace and equivalent Unicode typography only when meaning is
    preserved. Preserve meaningful case, symbols, and units (e.g. T versus t,
    N versus n, mm versus cm, cm2 versus cm4); do not lowercase table symbols or
    discard units to force a match. Cosmetic line wrapping is not a new column.

  For "kv" nodes:
    A header row is one that contains at least ONE key label from the node's fields as
    a complete normalised token or contiguous token sequence.

  For kv labels, compare case-insensitively and strip surrounding punctuation
  (‘:‘, ‘.‘, ‘,‘, ‘#‘). The stricter table rules above govern table headers.

STEP 4 -- Assign content rows to blocks
  Process pages in ascending page order (the script can therefore carry state from one
  page to the next). After locating the header row(s) of each block on the page:
    - Include the validated header and its body within that table's column extent.
      Detect boundaries of ALL candidate tables/blocks, including rejected or unknown
      ones. Stop before a new candidate, incompatible header, section boundary, or
      incompatible column structure. A rejected table must never be absorbed into
      the preceding accepted table or kv block. Keep unknown regions unassigned.
    - Use validated column spans and local geometry to distinguish side-by-side
      tables. Vertical proximity alone must not assign words between different tables.
    - Preserve valid wrapped rows, blank cells, and bottom data rows inside the
      accepted table. Sparse rows alone are not evidence of a new table. Noise rows
      stripped in Step 2 are skipped, but do not terminate the current table.
    - CONTINUATION BLOCKS: rows that appear ABOVE the first header band on a page (or
      on a page with no header band at all) may continue the immediately preceding
      page's fully validated table ONLY with affirmative evidence that the entire
      column structure and body-row layout remain compatible. Partial column overlap
      is insufficient. There must be no intervening section boundary, unknown table,
      or rejected header. Any visible repeated header must pass the full Step 3
      validation; a failed header match is NEVER a headerless continuation. Clear
      continuation state on rejection or uncertainty. If continuity cannot be proved,
      leave the region unassigned. Emit validated continuations as separate page-local
      blocks of the same node, preserving the established signature.

---------------------------------------------------------------
PROHIBITED HEURISTICS -- these overfit the sample pages and break on unseen pages
---------------------------------------------------------------

The script will run on OTHER pages of the same document family, not just the sample
CSV. The following shortcuts fit the sample but silently drop data later. Do NOT:

  - Suppress, filter, or skip kv blocks based on page position or page index
    (e.g. "kv blocks only appear on the first page", "drop kv candidates on
    continuation pages"). A kv node can appear on ANY page -- once per record --
    and records start mid-document.
  - Filter blocks based on HOW MANY blocks of a type were detected on a page
```

```
    (e.g. "if fewer than 2 kv nodes matched, discard all kv candidates").
    Detect each block independently on its own evidence.
  - Assume a fixed number of records or blocks per page, or that all template
    nodes appear on every page / before a new record starts.
  - Let continuation handling discard a page's own header-band detections:
    continuation logic only applies to rows ABOVE the first header band; it must
    never remove or override blocks whose headers were found on the page.
  - Hard-code absolute y-coordinates or page numbers from the sample CSV.
  - Build blocks that span pages. Blocks are PAGE-LOCAL (this is validated after
    the script runs and a violation is a hard failure): a block emitted under
    page P must contain ONLY page-P words. When a table's rows continue onto the
    next page, emit a separate continuation block of the same node on that page
    (Step 4) -- never absorb later pages' words into the header page's block, and
    never synthesize a global cross-page y-axis by stacking page heights.

A candidate with missing/extra fields, incompatible layout, ambiguous identity, or
insufficient evidence emits no block for that node. Do not coerce it into a known
template. A page with no accepted blocks must still appear with an empty blocks list.
Print concise rejection diagnostics with page, candidate node_id, and reason
(missing/extra field, unit mismatch, column order, hierarchy, layout, or ambiguity).
Do not add rejected regions to the block JSON under fabricated node IDs.

Before finishing generation, exercise the matching logic on sample-derived cases:
  - Accept the complete reference and translations/uniform scalings of it, with
    small coordinate jitter and harmless within-cell wrapping.
  - Reject removal/addition/duplication of a column, swapped columns, changed units,
    regrouped headers, and the same words scattered into unrelated columns/body rows.
  - Reject a table with an extra imperial mass column even when every expected
    metric field is still present.
  - Ensure a rejected table below a valid block is not swallowed by that block,
    and a rejected repeated header cannot be rescued by continuation handling.
These are offline checks; do not install packages or call external services.

STEP 5 -- Output
  Write `<out_path>/block_separation.json`:
  {
    "pages": [
      {
        "page": <int>,
        "blocks": [
          {
            "uid": <int>,
            "block_id": "<node_id>_<instance>",
            "type": "<table|kv>",
            "words": [
              {"id": <int>, "x0": <float>, "top": <float>, "x1": <float>, "bottom": <float>,
     "text": <str>},
              ...
            ]
          },
          ...
        ]
      },
      ...
    ]
  }

  `block_id` is the template node_id followed by an underscore and the 0-based count
  of that node's blocks on the page (e.g. the second block of node 1 on a page is
  "1_1"). The counter resets on every page.

  `uid` is a globally unique integer that increments by 1 for every block across all
```

```
  pages, starting from 0.

  Print a summary line per page to stdout: page number, number of blocks found,
  total words assigned.

The script must also contain:

  if __name__ == "__main__":
      import sys
      main(sys.argv[1], sys.argv[2])

Use only the Python standard library (csv, json, os, re, collections). Do not install
any packages. Always open files with encoding="utf-8", errors="replace".
```

A.4 Block extraction

```
You are inside the folder containing word-level OCR samples for block type "2".

The template node that describes this block type is:
{
  "type": "table",
  "node_id": 2,
  "bid": [
    1
  ],
  "child": -1,
  "fields": [
    "Item",
    "Length"
  ],
  "complexity": {
    "same_row_structure": true,
    "nested": false
  }
}

The "fields" list above names every field present in this block. Use these names verbatim
as keys in the output (do not rename or invent fields).

The "complexity" object describes the structural difficulty of this block and must guide
how you write the extraction logic:
- For kv blocks:
  - "repeated": true -> the block appears more than once inside a single record; your
    extract() must return "data" as a list of dicts, one per repetition.
    CRITICAL: a new repetition begins ONLY when the FIRST field in the template
    "fields" list reappears in a new row. Do NOT start a new repetition when any
    other (non-first) field is seen again -- those appearances are continuation lines
    of the current repetition, not a new one. false -> return "data" as a single dict.
- For table blocks:
  - "same_row_structure": true -> every data row has the same layout (one value per
    column); a simple row-grouping by top-coordinate is sufficient. false -> rows may
    differ structurally (spanning cells, blank separators, sub-labels); your logic must
    handle the variation rather than assuming uniform alignment.
  - "nested": true -> a sibling/child template node appears spatially within this
    block's page region (the template "child" field points to it). This is a layout
    property only: it does NOT mean cell values are structured. Every cell in THIS
    block must still be extracted as a plain string -- do NOT produce dicts, lists, or
    any non-string cell values, regardless of what punctuation (colons, commas, etc.)
```

```
    appears inside the cell text. The child node has its own extraction script.
    false -> each cell contains a single plain value; extract it directly.

Each .json file in this folder is one sampled instance of this block type and has the
    structure:
  {
    "page": <int>,
    "block_id": "<template_id>_<instance>",
    "type": "table",
    "words": [
      {"x0": <float>, "top": <float>, "x1": <float>, "bottom": <float>, "text": <str>},
      ...
    ]
  }

Word coordinates are relative to the top-left corner of the block (top-left = 0, 0).
The corresponding .png files show the visual rendering of each sample.

Your task:
1. Read and analyse ALL the .json (and optionally .png) sample files in this directory
   to understand the spatial layout of the block.
2. Write a Python script and save it as 2.py in the current directory.

The script must define exactly this function:

  def extract(block: dict) -> dict

  - `block` is a dict with keys "block_id" (str), "type" (str), and "words" (list[dict]).
    Each word dict has "x0", "top", "x1", "bottom", "text" (coordinates relative to the
    block's top-left corner).
  - The function reads block["block_id"] to populate the "block_id" field of the output.

Return format for a TABLE block ("table" == "table"):
  {
    "type": "table",
    "block_id": "<block_id>",
    "data": [
      {"<field_1>": "<value row 1>", "<field_2>": "<value row 1>", ...},
      {"<field_1>": "<value row 2>", "<field_2>": "<value row 2>", ...},
      ...
    ]
  }
  Each entry in "data" is one data row. Use spatial clustering to identify rows and to
  align each cell with the correct column header from the template fields list.

Return format for a KV block ("table" == "kv"):
  Single KV group:
  {
    "type": "kv",
    "block_id": "<block_id>",
    "data": {"<field_1>": "<value_1>", "<field_2>": "<value_2>", ...}
  }
  Multiple KV groups inside the same block:
  {
    "type": "kv",
    "block_id": "<block_id>",
    "data": [
      {"<field_1>": "<value_1>", ...},
      {"<field_1>": "<value_2>", ...},
      ...
    ]
  }
```

```
  Determine whether "data" should be a dict or a list by inspecting the samples.

How `extract()` is called at runtime:
  The pipeline (pipeline.py) has already run a separate block-separation step that
  locates every block instance on every page and relativises its word coordinates.
  For each detected instance it calls:

    relative_block = {**block, "words": _relativize(block["words"])}
    result = extract(relative_block)

  So `extract()` always receives a single, already-isolated block whose word
  coordinates start at (0, 0). It must NOT attempt to locate the block inside a
  full page, parse an OCR CSV, or do any document-level logic -- all of that is
  handled upstream. Focus entirely on parsing the words of the one block passed in
  and returning the structured output.

Additional requirements:
- Use the template "fields" as the canonical key names in all output dicts.
- Use word positions (x0, top, x1, bottom) to group words into rows/columns and to
  match labels to values. Do not hard-code pixel thresholds; derive them from the data.
- COLUMN BOUNDARIES (table blocks): derive the column x-centers from the header
  band of THE BLOCK BEING EXTRACTED, at call time -- locate each field label's
  words inside `block["words"]` and use their positions as the column anchors.
  Never bake in x-coordinates measured on the samples: unseen pages shift
  horizontally, and stale anchors silently swap adjacent columns (values land
  under the wrong field, which is worse than a missing value). If the block has
  no header band of its own (a continuation block), fall back to clustering the
  data words by x-position into as many columns as the template has fields, and
  assign columns to fields left-to-right in template order.
- After assigning cells, sanity-check the column mapping: the detected columns'
  left-to-right order must match the template "fields" order (when the labels are
  present). If a value row systematically fills a column whose header words were
  never found, prefer the positional (left-to-right) assignment.
- Row-filter guards: do not reject data rows with strict regexes on token counts
  or exact value formats (e.g. "row must have >= 9 tokens", "first token must be
  a pure integer", "tokens 2-3 must match a date pattern"). OCR on unseen pages
  produces merged/split tokens and o/0 confusions; strict filters silently drop
  every row. Prefer positional evidence (a word under the first column's anchor)
  and treat format checks as soft signals only.
- Be generalizable: the function must work correctly on any instance of block type
  "2", not only the sampled ones.
- Use only the Python standard library. Do not import any external packages.
- Define ONLY the `extract(block: dict) -> dict` function. Do NOT write a `main()`
  function, a CLI entry point, or any block-detection / document-parsing logic.

Save the finished script as 2.py in the current directory.
```

A.5 Optional difficulty routing: system instructions

```
You are a classification assistant for a PDF data extraction pipeline.
You will receive sample OCR layouts for a single block type (a recurring section of a PDF
    document that appears once or more per page).

Your task is to decide whether extracting this block requires a powerful model (HARD) or
    whether a cheaper, smaller model is sufficient (EASY).

There are exactly two reasons to classify a block as HARD:
```

```
  1. VARIABLE STRUCTURE -- the block contains a section or sub-group that repeats a
     different number of times across samples. For example, a line-item table with 3 rows on
     one page and 7 on another, or an optional sub-section that appears in some samples but
     not others.

  2. MULTI-LINE VALUES -- at least one field has a value that wraps across multiple lines in
      any sample (e.g., a long address, a free-text description).

If NEITHER of those is true -- even if values differ between samples, even if there are
    minor positional shifts -- classify the block as EASY.

Respond with ONLY a JSON object on a single line, e.g.:
{"difficulty": "easy", "reason": "flat fixed structure, all values single-line"}
or
{"difficulty": "hard", "reason": "line-item table row count varies between samples"}
```

A.6 Training reference annotations

```
You are a data-extraction assistant. You will receive one or more document pages as layout-
    preserving OCR. Each line includes y coordinates and each word includes its x span. Use
    these coordinates to reconstruct rows, columns, and key/value alignment.
Extract every physical data block visible on those pages according to the template.

TEMPLATE
Every block must correspond to one of these nodes. Use the node's fields as the
exact key names; do not rename, invent, or omit fields.

[
  {
    "type": "kv",
    "node_id": 0,
    "bid": [
      0
    ],
    "child": -1,
    "fields": [
      "Record ID",
      "Date"
    ],
    "complexity": {
      "repeated": false
    }
  },
  {
    "type": "table",
    "node_id": 2,
    "bid": [
      1
    ],
    "child": -1,
    "fields": [
      "Item",
      "Length"
    ],
    "complexity": {
      "same_row_structure": true,
      "nested": false
    }
```

```
  },
  {
    "type": "table",
    "node_id": 3,
    "bid": [
      1
    ],
    "child": -1,
    "fields": [
      "Item",
      "Width",
      "Height"
    ],
    "complexity": {
      "same_row_structure": true,
      "nested": false
    }
  }
]

ALTERNATIVE RECORD SLOTS
The following virtual XOR nodes describe mutually exclusive layouts at one position within a
     logical record:
[
  {
    "type": "xor-table",
    "node_id": 1,
    "includes": [
      2,
      3
    ]
  }
]
An xor-table includes table node IDs; an xor-kv includes KV node IDs. A record satisfies
    that slot when any one included physical node appears. Use this to distinguish record
    boundaries and identify the correct physical layout. Never emit an XOR node as a block,
    and never merge alternative physical nodes into one block. The required output remains
    the flat blocks list below; deterministic record grouping runs after the blocks are
    submitted.


Each template node has:
  "node_id": identifier to use verbatim as "block_id"
  "type": "table" or "kv"
  "fields": canonical field or column names

PHYSICAL BLOCK RULES
- Do not group blocks into logical records.
- Emit blocks in page order and top-to-bottom order within each page.
- Every block is page-local and must have the physical 1-based page number in
  its "page" field.
- If the same template node appears more than once, emit every occurrence as a
  separate block even when occurrences are consecutive.
- If a table continues onto another page, emit one table block per page. Never
  concatenate rows across pages or across repeated table occurrences.
- Reprinted kv/header blocks are separate physical blocks when they appear on
  separate pages.

OUTPUT FORMAT
Reply with a single JSON object and nothing else:

{
```

```
  "blocks": [
    {
      "page": 1,
      "type": "table",
      "block_id": "<node_id>",
      "data": [
        {"<field_1>": "<value>", "<field_2>": "<value>"}
      ]
    },
    {
      "page": 1,
      "type": "kv",
      "block_id": "<node_id>",
      "data": {"<field_1>": "<value>", "<field_2>": "<value>"}
    }
  ]
}

Rules:
1. Root has only "blocks" (list); there are no records.
2. Each block has "page", "type", "block_id", and "data".
3. Keys in every data dict exactly match the corresponding template fields.
4. Table data is a list of row dicts; kv data is one dict.
5. Skip headers, footers, page numbers, and metadata that do not match a node.
6. Copy text exactly as printed. When OCR is provided, its exact character
   sequences are authoritative; do not correct or normalize them.
7. Use empty string "" for a field with no visible value.
8. Output only the JSON object, without prose or markdown fences.
```

A.7 Test reference annotations

```
You are a data-extraction assistant. You will receive one or more document page images and
    the exact OCR word list for each page. Use the images for layout and the OCR text as the
     authoritative character source.
Extract every physical data block visible on those pages according to the template.

TEMPLATE
Every block must correspond to one of these nodes. Use the node's fields as the
exact key names; do not rename, invent, or omit fields.

[
  {
    "type": "kv",
    "node_id": 0,
    "bid": [
      0
    ],
    "child": -1,
    "fields": [
      "Record ID",
      "Date"
    ],
    "complexity": {
      "repeated": false
    }
  },
  {
    "type": "table",
```

```
    "node_id": 2,
    "bid": [
      1
    ],
    "child": -1,
    "fields": [
      "Item",
      "Length"
    ],
    "complexity": {
      "same_row_structure": true,
      "nested": false
    }
  },
  {
    "type": "table",
    "node_id": 3,
    "bid": [
      1
    ],
    "child": -1,
    "fields": [
      "Item",
      "Width",
      "Height"
    ],
    "complexity": {
      "same_row_structure": true,
      "nested": false
    }
  }
]

ALTERNATIVE RECORD SLOTS
The following virtual XOR nodes describe mutually exclusive layouts at one position within a
     logical record:
[
  {
    "type": "xor-table",
    "node_id": 1,
    "includes": [
      2,
      3
    ]
  }
]
An xor-table includes table node IDs; an xor-kv includes KV node IDs. A record satisfies
    that slot when any one included physical node appears. Use this to distinguish record
    boundaries and identify the correct physical layout. Never emit an XOR node as a block,
    and never merge alternative physical nodes into one block. The required output remains
    the flat blocks list below; deterministic record grouping runs after the blocks are
    submitted.


Each template node has:
  "node_id": identifier to use verbatim as "block_id"
  "type": "table" or "kv"
  "fields": canonical field or column names

PHYSICAL BLOCK RULES
- Do not group blocks into logical records.
- Emit blocks in page order and top-to-bottom order within each page.
```

```
- Every block is page-local and must have the physical 1-based page number in
  its "page" field.
- If the same template node appears more than once, emit every occurrence as a
  separate block even when occurrences are consecutive.
- If a table continues onto another page, emit one table block per page. Never
  concatenate rows across pages or across repeated table occurrences.
- Reprinted kv/header blocks are separate physical blocks when they appear on
  separate pages.

OUTPUT FORMAT
Reply with a single JSON object and nothing else:

{
  "blocks": [
    {
      "page": 1,
      "type": "table",
      "block_id": "<node_id>",
      "data": [
        {"<field_1>": "<value>", "<field_2>": "<value>"}
      ]
    },
    {
      "page": 1,
      "type": "kv",
      "block_id": "<node_id>",
      "data": {"<field_1>": "<value>", "<field_2>": "<value>"}
    }
  ]
}

Rules:
1. Root has only "blocks" (list); there are no records.
2. Each block has "page", "type", "block_id", and "data".
3. Keys in every data dict exactly match the corresponding template fields.
4. Table data is a list of row dicts; kv data is one dict.
5. Skip headers, footers, page numbers, and metadata that do not match a node.
6. Copy text exactly as printed. When OCR is provided, its exact character
   sequences are authoritative; do not correct or normalize them.
7. Use empty string "" for a field with no visible value.
8. Output only the JSON object, without prose or markdown fences.
```

A.8 Manual reference and completeness audit

```
You are a data-extraction assistant. You will receive one or more document page images and
    the exact OCR word list for each page. Use the images for layout and the OCR text as the
     authoritative character source.
Extract every physical data block visible on those pages according to the template.

TEMPLATE
Every block must correspond to one of these nodes or a justified evaluation-only addition
    below. Use the node's fields as the
exact key names; do not rename, invent, or omit fields.

[
  {
    "type": "kv",
    "node_id": 0,
```

```
    "bid": [
      0
    ],
    "child": -1,
    "fields": [
      "Record ID",
      "Date"
    ],
    "complexity": {
      "repeated": false
    }
  },
  {
    "type": "table",
    "node_id": 2,
    "bid": [
      1
    ],
    "child": -1,
    "fields": [
      "Item",
      "Length"
    ],
    "complexity": {
      "same_row_structure": true,
      "nested": false
    }
  },
  {
    "type": "table",
    "node_id": 3,
    "bid": [
      1
    ],
    "child": -1,
    "fields": [
      "Item",
      "Width",
      "Height"
    ],
    "complexity": {
      "same_row_structure": true,
      "nested": false
    }
  }
]

ALTERNATIVE RECORD SLOTS
The following virtual XOR nodes describe mutually exclusive layouts at one position within a
     logical record:
[
  {
    "type": "xor-table",
    "node_id": 1,
    "includes": [
      2,
      3
    ]
  }
]
An xor-table includes table node IDs; an xor-kv includes KV node IDs. A record satisfies
    that slot when any one included physical node appears. Use this to distinguish record
```

```
    boundaries and identify the correct physical layout. Never emit an XOR node as a block,
    and never merge alternative physical nodes into one block. The required output remains
    the flat blocks list below; deterministic record grouping runs after the blocks are
    submitted.


Each template node has:
  "node_id": identifier to use verbatim as "block_id"
  "type": "table" or "kv"
  "fields": canonical field or column names

PHYSICAL BLOCK RULES
- Do not group blocks into logical records.
- Emit blocks in page order and top-to-bottom order within each page.
- Every block is page-local and must have the physical 1-based page number in
  its "page" field.
- If the same template node appears more than once, emit every occurrence as a
  separate block even when occurrences are consecutive.
- If a table continues onto another page, emit one table block per page. Never
  concatenate rows across pages or across repeated table occurrences.
- Reprinted kv/header blocks are separate physical blocks when they appear on
  separate pages.

OUTPUT FORMAT
Reply with a single JSON object and nothing else:

{
  "blocks": [
    {
      "page": 1,
      "type": "table",
      "block_id": "<node_id>",
      "data": [
        {"<field_1>": "<value>", "<field_2>": "<value>"}
      ]
    },
    {
      "page": 1,
      "type": "kv",
      "block_id": "<node_id>",
      "data": {"<field_1>": "<value>", "<field_2>": "<value>"}
    }
  ]
}

Rules:
1. Root has "blocks" (list) and optionally "template_extension"; there are no records.
2. Each block has "page", "type", "block_id", and "data".
3. Keys in every data dict exactly match the corresponding template fields.
4. Table data is a list of row dicts; kv data is one dict.
5. Skip headers, footers, page numbers, and metadata that do not match a node.
6. Copy text exactly as printed. When OCR is provided, its exact character
   sequences are authoritative; do not correct or normalize them.
7. Use empty string "" for a field with no visible value.
8. Output only the JSON object, without prose or markdown fences.

MANUAL TEMPLATE COMPLETENESS AUDIT
Inspect every page for structured regions missing from the inferred template.
Apply TWIX's template inference methodology: K headers with aligned V data rows
form tables; self-contained label/value KV rows form KV blocks; M titles,
footers, disclaimers, section labels and free text are metadata, never new nodes.
Rare or conditional structured blocks count, even if visible only once. Require
```

```
visible fields AND values. Reuse existing nodes for repeated occurrences,
continuations, wrapped/partial headers, and different values of the same fields.
Do not invent fields or values or create nodes from unexplained whitespace.
Use unique fields, "Group - Column" for tiered headers and deterministic " (2)"
suffixes for genuine duplicate columns. Preserve image layout and exact OCR.

Only when necessary, add "template_extension" to the response:
{"template_extension": {"template": [<complete expanded ordered template>],
 "evidence": [{"node_id": <new ID>, "pages": [<local test pages>],
 "reason": "Visible labels and aligned values; why no existing node fits;
 natural record position; optional/repeated/alternative layout justification"}]}}
Keep every existing physical node unchanged, including IDs, fields and bids.
Keep original nodes in their relative order; insert additions in their natural
record position. New IDs are fresh integers above ALL original node IDs; never
renumber existing nodes. New physical nodes require type, fields, node_id, bid,
child (-1 for siblings/leaves), and complexity (tables: same_row_structure and
nested booleans; KV: repeated boolean). Derive bids from visual record position.
If an existing node would need modification, reuse it only if its exact fields
fit; otherwise add a genuinely distinct layout, never silently change it.
For mutually exclusive alternatives at the same record position use a matching
xor-table/xor-kv slot and matching bids. Existing XOR includes may only expand;
a physical node belongs to one XOR slot at most. New XOR nodes need at least two
physical members and evidence explaining mutual exclusion; never emit XOR blocks.
Independent optional blocks that can coexist are not XOR alternatives.
Provide evidence for EVERY added node, including virtual XOR slots. Every page
cited for a physical node must contain an extracted non-empty block of that node.
Return all physical blocks against the expanded template in the ordinary flat
format. When no addition is justified, return only "blocks" as usual.
The expanded template is used ONLY for held-out evaluation, symmetrically for
truth and candidate grouping; it does not retrain or repair the learned pipeline.
```

A.9 Optional training repair

```
You are debugging a PDF data-extraction pipeline. Fix the Python scripts in the current
    directory so the pipeline output better matches the ground truth.

PIPELINE-HEALTH VIOLATIONS  (from train_eval/pipeline_health.json)
================================================================
These are HARD failures -- the run is rejected while any of them remains, no
matter how good the accuracy looks:
  - <PIPELINE_HEALTH_PROBLEM_IF_PRESENT>


PIPELINE OVERVIEW
=================
The pipeline runs in this order:
1. block_separation.py <ocr.csv> <out_dir>
   Detects typed blocks (kv, table) within the OCR data.
   Writes: out_dir/block_separation.json

2. extraction_scripts/<id>.py  -- each defines  extract(block: dict) -> dict
   Extracts field values from one physical block.
   Called by pipeline.py on every block produced by block_separation.

3. record_grouping.py
   Deterministically traverses extracted template node ids and writes grouped.json.
   It is fixed pipeline code and is not generated or repairable.
```

```
EVALUATION ERRORS  (from train_eval/eval.txt)
==============================================
<TRAIN_EVALUATION_REPORT>

FILES YOU MAY MODIFY
====================
  - block_separation.py
  - extraction_scripts/0.py
  - extraction_scripts/2.py
  - extraction_scripts/3.py

DO NOT modify pipeline.py, template.json, or any other file.

PRIORITY FIXES BY ERROR CODE
============================
[R-RECORD-MISSING]
  A logical record has no candidate counterpart. Usually a required boundary
  block was missed; fix block_separation.py so the template-node sequence is
  complete and ordered.

[R-RECORD-EXTRA]
  A spurious logical record was formed. Check block_separation.py for a
  false-positive or misordered block that creates an artificial restart.

[R-NODE-MISSING]
  A block type was not detected on some pages.
  Fix block_separation.py: improve header-row detection or word-assignment logic.

[R-NODE-EXTRA]
  Spurious extra blocks are being emitted.
  Fix block_separation.py or the relevant extraction script -- add a guard so the
  rule only fires when the block clearly exists on the page.
  NOTE: per-page continuation blocks and consecutive occurrences of the same
  node remain separate by design. NEVER absorb several physical blocks into one.

[R-CELL-DIFFERS] / [R-KV-VALUE-DIFFERS]
  Field values are extracted incorrectly.
  Fix the relevant extraction_scripts/<id>.py -- check column alignment, label/value
  splitting, and text concatenation logic.

ANTI-OVERFITTING RULES  (MANDATORY -- violations cause test-set failures)
=========================================================================
1. NEVER hard-code specific text values, names, amounts, dates, or any content from
   the training pages. All logic must be based on positions and field label patterns.
2. NEVER hard-code absolute pixel/coordinate thresholds. Derive all spatial thresholds
   dynamically from the data being processed (medians, percentages, inter-word gaps).
3. NEVER force output counts (blocks or rows) to match the training-data counts.
   Counts must emerge from the algorithm -- never be hard-coded.
4. NEVER branch on the document name, page-number values, or any fixed metadata.
5. Every modified script must work correctly on pages it has never seen.
6. Blocks are PAGE-LOCAL (validated after every run; a violation rejects the run):
   a block emitted under page P in block_separation.json may contain ONLY page-P
   words. Table rows continuing onto later pages must be emitted as separate
   continuation blocks of the same node on each page. They remain separate in
   extracted.json. Never absorb later pages' words into one block and never
   synthesize a global cross-page y-axis.
7. NEVER suppress or drop kv blocks based on page position/index or on how many
   blocks of a type were detected on a page.

Read the existing scripts first to understand the current logic, then make targeted
fixes that address the errors listed above without breaking what already works.
```

A.10 Baseline generation skill

```
---
name: pdf-extraction-baseline
description: Create a data template and standalone Python PDF extractor from five supplied
    pages. Use when the extractor must process complete PDFs of the same document family.
---

# PDF extraction baseline

Identify the document folder containing the five supplied PDF pages. It may be the current
    folder or a document subfolder when invoked from its parent. Work autonomously from
    those pages, whether they are in one PDF or several PDFs. Inspect all five pages before
    designing the extraction. Make reasonable assumptions where needed and finish without
    asking the user questions.

Create every deliverable inside that document folder: ‘template.json‘, ‘extract.py‘,
    dependency information if needed, and any output from the validation run. Do not place
    generated files in the parent folder merely because that is the current working
    directory.

Infer the data structure from those pages and write ‘template.json‘ describing the fields,
    nesting, and repeated items to extract. Then write a standalone Python script, ‘extract.
    py‘, that accepts an input PDF path and an output JSON path, processes every page of the
     input, and writes data organized according to the template. The script must use
    ordinary Python code and, if needed, conventional PDF parsing libraries. It must not
    call or load an LLM or any other model, use a remote service, or require an agent when
    it runs. Do not hard-code sample values or a five-page limit. Include dependency
    information if the script needs third-party packages.

Run the script against all five supplied pages, writing its test output inside the document
    folder. Check that it completes and produces valid JSON consistent with the template.
    Fix observed failures. Leave all generated files in the document folder and give the
    user a concise command for running the script on a PDF.
```

A.11 Baseline evaluation and adaptation skill

```
---
name: pdf-extraction-eval-adapter
description: Adapt and evaluate a completed PDF extractor on the 20 pages after its five
    development pages, with a Codex-generated reference and a JSON result.
---

# PDF extraction baseline evaluation

Use only after the independent baseline generator has finished. The user supplies the
    complete source PDF and the folder containing the generated extractor, template, and
    five-page sample. Work autonomously once those paths are available. Put all evaluation
    files in a separate workspace. Do not modify the baseline extractor, template, sample
    output, or generation skill. Codex generates the reference from local PDF images and
    text; do not call OpenRouter or another external model service.

## Select the evaluation pages

Identify the five physical source pages used to generate the extractor and verify their
    mapping to the complete PDF. Evaluate the **next 20 available physical pages after those
     five**, excluding every development page. For the usual first-five-pages sample, this
```

```
    means pages 6-25. Do not substitute a spread-out benchmark split. If fewer than 20 pages
     remain, use all remaining pages and record the shortfall; if none remain, record a
    failure. Ground truth must cover only these selected evaluation pages, not the entire
    document.

## Run the frozen extractor

Record the full PDF's page count and hash, plus hashes of the extractor and its template.
    Run the exact generated script on the **entire** PDF first; save its raw output or exit
    status, stdout/stderr, and duration. A full-document failure remains a reported failure
    even when page-level scoring can continue.

For the selected evaluation pages, use the full output if it can be restricted to those
    pages without ambiguity. Otherwise run the same frozen script on a PDF containing those
    20 pages. If that run fails, isolate the selected pages and run the same script on each
    one; a failed page contributes an **empty prediction**, while successful pages retain
    all extracted content. Record every page's source number, exit status, extracted row/
    block count, and error text. Keep page selection and failure handling in evaluation-only
     code; never repair or retrain the baseline script.

## Create and freeze the adapter

Inspect the baseline template and five-page output, the development PDF for printed
    representation details, the raw candidate's keys/types, and the benchmark template/
    scorer contract. Write an evaluation-only `adapter.py` with `adapt(raw) -> dict` and a
    CLI for raw JSON to scorer JSON. Its runtime is deterministic plain Python: no PDF,
    model, network, or reference access. Preserve raw output. Write a machine-readable
    mapping manifest that accounts for every source field: target field and conversion,
    extra prediction, or explicit exclusion as provenance/out-of-scope metadata. Never
    silently drop substantive fields, rows, or failed conversions or invent missing values.

For TWIX, inspect the current converter, scorer, and record-grouping contract. Emit
    canonical records/nodes directly; keep physical page-local nodes or logical merged nodes
     **consistent for candidate and reference**. Do not let its converter treat an unknown
    baseline format as empty. Validate row order, page provenance, all field mappings,
    deterministic repeatability, and display-format conversions against the development
    pages. Save adapter and manifest hashes before preparing ground truth. If a later
    contract correction is unavoidable, preserve and score the original version, document
    the correction and its evidence, and label the revised accuracy separately.

## Generate the reference locally

Codex inspects **every selected page image and its PDF text/OCR**. Use the baseline and
    benchmark templates as starting descriptions, not closed field lists. Inventory all
    substantive tables, key-value blocks, rows, and fields, including structures absent from
     the five development pages or missed by the extractor. Do not use baseline predictions,
     adapted output, scores, or an existing saved reference to generate ground truth. Local
    parsing tools may assist transcription, but visually check the page structures and
    verify per-page row/field coverage.

Write an evaluation-only `evaluation_template.json`; add verified new nodes or fields with
    stable IDs while preserving the scorer's template contract. Do not alter original
    templates. Produce raw and canonical ground truth for all selected pages, including
    pages whose baseline extraction failed. Revisit any page affected by a template addition.
     Save the page inventory, template additions, transcription, and QA notes. Describe this
     as a Codex-generated, model-assisted reference, not human-verified truth or an external-
    model result.

## Score and always write `evaluation_result.json`

Load candidate and reference with the scorer's duplicate-key-safe loader where available,
    run the existing scorer unchanged, and record its version or source hash. Score only the
```

```
     selected pages. If another system is compared, use exactly the same pages, reference,
    node representation, and scorer version.

Create ‘evaluation_result.json‘ in the evaluation workspace at the start and update it after
     every phase using atomic writes. Leave a valid final JSON file even when extraction,
    adaptation, reference generation, or scoring fails. It must contain at least:

‘‘‘json
{
  "status": "failed",
  "accuracy": null,
  "accuracy_scale": "0_to_1",
  "cost": null,
  "pages": {"development": [], "evaluation": [], "requested": 20, "evaluated": 0},
  "full_document_run": {"status": "not_run", "exit_code": null, "reason": null},
  "candidate": {"successful_pages": 0, "empty_failed_pages": [], "rows_or_items": 0},
  "reference": {"source": "Codex-generated local PDF reference", "rows_or_items": 0, "
    template_additions": []},
  "score_components": {},
  "errors": [],
  "artifacts": {},
  "hashes": {}
}
‘‘‘

Always include the top-level ‘cost‘ field and initialize it to ‘null‘. Leave it
for the user to fill manually; do not calculate, infer, or overwrite it.

Use ‘completed‘, ‘partial‘, or ‘failed‘ for final ‘status‘. Each ‘errors‘ entry must have ‘
    phase‘, ‘page‘ (or ‘null‘), ‘what_happened‘, ‘cause‘, ‘effect‘, and ‘evidence‘. Use a
    numeric ‘accuracy‘ only after a valid score; use ‘null‘ and explain why if scoring could
     not run. ‘status‘ is ‘partial‘ when the full run or individual pages fail but a valid
    held-out score exists. Include an error entry for each failed evaluation page and for
    any full-run or later-phase failure. In each entry, state what happened, the supported
    cause (or ‘unknown‘), its effect on the score, and concise evidence such as the
    exception or stderr. Summarize error counts when useful, but retain the individual page
    failures. Include paths to raw output, adapter, reference, scorer report, and page log
    in ‘artifacts‘.

At the end of every evaluation, also copy the final summary JSON and the human-readable
    scorer text report into the corresponding baseline document folder, for example ‘
    baseline/<doc_id>/evaluation_result.json‘ and ‘baseline/<doc_id>/scorer_report.txt‘.
    These two files are stable comparison artifacts and should be present even when the
    richer evaluation workspace lives elsewhere. Do not move or overwrite the baseline
    extractor, template, development PDF, sample output, or generation skill. If scoring
    cannot run, still save the final ‘evaluation_result.json‘ in the baseline folder; save
    the scorer text report there only when it exists, and record its absence in ‘errors‘ or
    ‘artifacts‘ as appropriate.

Report the accuracy and main failure causes to the user, with a link to the baseline-folder ‘
    evaluation_result.json‘ plus the evaluation workspace copy.
```

Appendix B

Performance Mismatch: Detailed Diagnostic Evidence

This appendix explains the source-document structures behind the failures discussed in section 4.1.1. Each case connects printed evidence to a rule in the frozen baseline program, then distinguishes the resulting execution failure or omission from the recorded extraction score. Page numbers refer to physical pages of the original PDF, not printed page labels or local page numbers in a replay. In the steel catalogue, one physical PDF page contains a two-page printed spread.

The report identifiers differ from original filenames: `id_01`, `id_03`, `id_07`, `id_21`, `id_22`, and `id_23` correspond respectively to `id_03.pdf`, `id_08_upscaled.pdf`, `id_60.pdf`, `id_49.pdf`, `new_01.pdf`, and `new_04.pdf`. The selected `id_23` baseline is the seven-page replacement run. Historical salary-listing probes concern a superseded `id_10` program and are separated below from its successful selected replacement.

PAN [Permanent Account Number, an Indian tax identifier] is a source field in the stakeholder register discussed below.

The analysis uses saved execution logs, frozen source code, targeted replay outputs, and inspection of the corresponding PDF text and geometry. Replay success establishes execution behavior, not independently verified accuracy. Some replay pages lie outside a recorded TWIX 2 test split, and its test references can differ from the baseline's. No diagnostic replay below replaces the saved scores or contributes feedback to construction.

B.1 Printed date placeholders: `id_01`

The source is a motion-ruling table with a filed-date column, case identifiers, ruling marks, and explanatory text. On source page 6, row 22 visibly prints `Unknown` in the date column. It occupies the same bounded cell position as ordinary dates, so the exception is present in the document rather than caused by missing OCR text. The other cells and neighboring

Document	Baseline test (%)	Empty test pages	Complete-PDF failure
id_01	67.02	8	Printed date placeholder
id_03	14.07	18	Sparse numeric row; fallback also loses year context
id_07	100.00	0	Identifier outside the assumed PAN pattern
id_21	81.61	3	Wrapped detail-table header
id_22	99.26	0	Repeated printed part number
id_23	7.89	15	Changed table-column structure
Total		44	Six complete-PDF failures

Table B.1: Saved outcomes for the six baseline programs that fail on their complete PDFs. Empty-page counts concern their selected evaluation pages. A failure elsewhere in a PDF does not automatically lower the selected-page score.

rows remain readable.

The baseline converts every filed-date value with `datetime.strptime` using the format `%m/%d/%Y`. Conversion of `Unknown` raises an exception before the invocation writes JSON. Isolated-page fallback therefore loses the complete page, including otherwise extractable rows. Eight selected pages fail, producing the recorded 67.02% accuracy. The failure is a mismatch between the printed field's permitted representations and the generated program's stronger date-validity assumption.

The TWIX 2 routine assigns fields geometrically and preserves strings without mandatory date conversion. Its source-page-6 replay returns all three rows, retaining `Unknown` beside two ordinary dates. The recorded TWIX 2 accuracy is 100%, but source page 6 is outside that saved test split. The replay supports a specific text-preservation mechanism rather than a new paired accuracy measurement.

B.2 Identifier validation and column shifts: `id_07`

This PDF is a long, regularly aligned stakeholder register. Source page 118 places the printed token `SIKKIMCATG` in the PAN column of row 9013, followed by `3,525` shares, `NA` [a printed missing-value placeholder] in the claimed column, and `12,161` admitted. Its position makes it an identifier-field value even though its text differs from the common PAN format. The same token also appears on other source pages, so treating it as an impossible value is unsafe for faithful transcription.

The baseline consumes a PAN token only when it matches five uppercase letters, four digits, and a final uppercase letter. When the match fails, the token remains at the front of the remaining cells and is passed to numeric conversion as the share count. Thus an identifier-

validity assumption changes column interpretation and aborts complete-document execution. All selected baseline test pages nevertheless succeed, yielding 100% recorded accuracy: the full-file failure occurs beyond that selected sample.

The TWIX 2 replay retains the token in the PAN column and keeps the neighboring numeric and placeholder values in their intended fields. It returns 77 rows from source page 118, which is outside the saved TWIX 2 test split. This case separates robustness over the complete document from accuracy on a limited sample; it does not demonstrate higher selected-page accuracy for TWIX 2.

B.3 Continuation context and blank numeric cells: `id_03`

The trade-statistics PDF repeats a table of monthly and annual observations with nine numeric measures. Year headings organize successive month rows, and a year's observations can continue onto the next physical page. Source page 6 begins with May before the next year heading appears. The missing heading is therefore preceding document context, not an absent month label or an unreadable table.

The baseline stores observations under a persistent `current_year`. Isolated-page fallback resets that state, so eighteen selected pages that begin with continuation months fail before producing output. Running the unchanged program on source pages 1 to 25 preserves the preceding context and returns 1,118 observations on the scored pages 6 to 25. This successful context probe was not rescored and does not replace the recorded 14.07% accuracy.

There is also an independent full-file failure. On the last source page, page 35, October, November, and December are printed as row labels beneath the populated September row, but their numeric cells are blank. The baseline requires nine numeric values for every recognized month and aborts at October. These blanks should not be inferred to be zero, nor should their presence make preceding observations unreadable. Source page 35 lies outside the baseline's scored pages 6 to 25.

The TWIX 2 program recovers page-local printed `Period` values and string cells without either a preceding-year requirement or a fixed populated-cell count. Its replay processes pages 6 and 35, retaining an October row with blank numeric values. This representation does not reconstruct year ownership for continuation months, and the replay also exposes footer contamination. Its 97.84% recorded accuracy therefore coexists with representational and content limitations. The comparison includes the interaction between output design, document context, and isolated-page fallback.

B.4 Changing table schemas within a fixed budget: `id_23`

The steel catalogue contains European, imperial, and metric beam tables. Its multi-level headers distinguish geometric dimensions from mechanical properties. The first seven source

pages cover the European schemas, whose mass values occupy one column. At source page 9, the imperial schema separates mass into `lb/ft` and `kg/m`, adding a column while retaining much of the surrounding header vocabulary. Later metric pages also introduce numeric section designations and cells left blank beneath shared designations.

The selected baseline receives source pages 1 to 7 and achieves 100% training accuracy through its existing adapter. The training audit checks all 171 designations and 4,446 numeric cells against local PDF transcription. Nevertheless, its generated geometry requires exactly 29 boundaries and a fixed gutter position. Source page 9 has 30 boundaries, triggering an exception before output is written. Pages 9 to 18 fail this check. Later pages fail a designation rule that requires an alphabetic prefix or rejects blank repeated cells. Only page 8 succeeds in isolated-page fallback; the fifteen other selected pages contribute empty predictions, producing 7.89% test accuracy.

The corresponding TWIX 2 sample contains source pages 1 to 5, 13, and 20, exposing the imperial and metric schemas within the same seven-page budget. Its template uses an XOR slot to organize three physical table alternatives, and generated separation code uses header and unit evidence to route blocks to their extractors. The saved replay returns 29 rows on source page 9 and 30 on page 19; recorded test accuracy is 99.09%. This case links the observed failure to missing variant evidence and restrictive generated rules. Sampling, representation, model choice, and specialized extraction change together, so their independent contributions are not isolated.

B.5 Wrapped invoice headers and repeated details: `id_21`

The invoice PDF combines address and metadata panels with detail tables containing line-item descriptions, flight periods, and individual aired spots. The discounted-rate variant prints `Rate` and `(Cash Discount Price)` on separate header lines. Several related rows can belong to one item, and both an invoice and an item can continue across physical pages. These are changes in header layout and detail cardinality, rather than merely longer cell values.

The baseline first locates a line beginning with `Line Channel Description`, then searches that same text line for all expected column labels. The wrapped rate label causes exceptions on source pages 7 to 9. Spacing changes on pages 10 and 11 instead cause the header search to fail and return a warning, leaving detail tables unextracted. The missed header also removes the stopping boundary used by address parsing; saved predictions on these pages contain table and footer text inside the address field. A single boundary-detection failure can therefore cause both omissions and contamination of another block.

The generated item representation introduces a separate loss mechanism. It stores one scalar set of flight-period fields and assigns incoming values only while those fields are empty. A later flight row for the same item is consequently consumed without preserving a second occurrence.

Successful execution on ordinary-rate pages does not establish that their repeated details are complete. Three selected pages produce empty predictions, while partial and silent omissions contribute to the recorded baseline accuracy of 81.61%.

The TWIX 2 sample includes the discounted-rate layout on cluster-selected source page 9. Its separator recognizes header tokens across tiers, and its physical-row representation preserves successive details. Replays detect tables on all seven inspected pages: 7, 8, 10, 11, 17, 18, and 19. Recorded accuracy is 91.60%, with remaining field and row-assignment errors. This is evidence for handling the inspected header and repetition patterns, not a guarantee of complete invoice reconstruction.

B.6 Repeated catalogue identifiers and unseen tables: `id_22`

The coupling catalogue repeats product tables beneath product headings and dimensional diagrams. The part number `T2750-0610` is visibly printed in tables on both source pages 46 and 47, with the same hose and thread measurements. Both occurrences are present in the PDF; they are not an OCR duplication introduced during extraction. An occurrence-preserving output must be able to retain a printed row even when its identifier has appeared earlier in the document.

The baseline applies one document-wide `seen_parts` set while validating all product items. Encountering the repeated part number raises an exception and prevents complete-file output. Its selected pages 6 to 25 contain neither occurrence and complete successfully, yielding 99.26% accuracy. The TWIX 2 replay appends rows within their physical blocks and preserves both occurrences. Pages 46 and 47 are outside its saved test split, so this demonstrates execution robustness rather than an increase in the recorded selected-page score.

Source page 90 reveals a different coverage problem. In addition to a coupling table, it contains a flange-dimensions table organized by nominal flange size and Code 61, Code 62, and Code 62C, with paired dimensions and units. Its field structure differs from the recurring part-number tables represented in the inferred template. Sonar assigns only 83 of 292 words to blocks on that page, or 28.4% coverage against a scan mean of 82.3%. Inspection connects this drop to the missing flange schema. The recorded TWIX 2 accuracy is 94.03%, below the baseline's selected-page score. Avoiding an invalid uniqueness constraint does not resolve incomplete template coverage.

B.7 Historical salary-listing probes and the selected replacement

Earlier diagnostic replays for `id_10` used the superseded `new_08` baseline. That program required a document-level jurisdiction heading and failed on isolated continuation pages; it also attempted numeric conversion of a printed asterisk placeholder on source page 9. The

PDF places `**********` in the maximum/control column of item `0776`, with its minimum cell blank. These observations concern an earlier program, not the selected baseline in the current results.

The selected `new_08_second_try` replacement completes the full PDF and records 100% training and test accuracy. Its saved output retains the asterisks in a separate printed-value field, leaves the numeric representation empty, and records a warning. The historical failure must therefore not be counted among the six complete-PDF failures or the 44 empty test pages in table B.1. This distinction prevents a successful replacement from inheriting an obsolete diagnosis.

B.8 Interpretation and reproducibility boundaries

The source evidence separates several mechanisms: visible values rejected by domain-validity rules, context lost during fallback, omitted or wrapped headers, repeated details reduced to scalar fields, and table schemas absent from the construction evidence. Templates, generated source, logs, page mappings, and raw predictions are needed to distinguish these causes. A training–test gap alone does not establish memorization, and a successful invocation does not establish complete or correct extraction.

The exported TWIX 2 runtime catches exceptions around individual block-extractor calls and can continue processing other blocks. This is failure containment rather than a completeness guarantee. A future context-preserving baseline evaluation could retain preceding pages while scoring only the held-out content; the successful context probes above are not substituted for such an evaluation.

Appendix C

Cost Accounting and Break-Even Inputs

This appendix gives the cost boundaries and per-document inputs used in figure 4.2.

C.1 Cost boundaries and reference-model proxy

Baseline fixed costs sum generation-session estimates, including three sessions for `id_05`; TWIX 2 fixed costs sum recorded template-inference, separator-generation, and extractor-generation calls. Reference generation, scoring, and baseline adaptation are excluded. Evaluation feedback did not revise the programs because repair was disabled. Separately logged auxiliary costs, such as start detection, are outside the recorded construction-phase totals and are not added retrospectively.

The direct-extraction cost per page is estimated from test-reference token usage when available, otherwise training-reference usage. The 21 documents with reference-token accounting use the latter. These measurements estimate the cost of direct LLM extraction; the expense of obtaining the measurements is excluded from pipeline construction cost. Input and output tokens are multiplied by the run's saved reference-model prices and divided by the sample page count. Most proxies use the GPT-5.4 reference configuration; `id_04`, `id_05`, `id_13`, and `id_23` use the recorded DeepSeek reference configuration. These are models used for the direct-extraction cost proxy, distinct from the GPT-6 Sol code-generation baseline. Price snapshots and workload densities differ across documents, so cross-document break-even differences cannot be interpreted as structural difficulty alone.

Baseline session costs use cache-aware estimates. TWIX 2 costs use input/output list prices without cache adjustments, as in the saved statistics. The generation-model price for `id_15` is absent from its snapshot after a pricing-fetch failure. Its existing statistics use a legacy fallback of USD 0.088606 per million input tokens and USD 0.177212 per million output tokens. We reproduce and flag that estimate rather than substitute a current price. The

reference-token counts remain missing for `id_06` and `id_14`. For these documents, full-document LLM-only extraction totals of USD 1.09 and USD 0.89 were supplied on 23 October 2026. Each PDF has 35 extraction pages, giving USD 0.031143 and USD 0.025429 per page. These totals supply only the direct-extraction denominator; they do not replace either system's construction cost. The actual model and token breakdown of the supplied totals are unverified.

C.2 Per-document inputs

In table C.1, G_B denotes baseline generation cost, G_T denotes TWIX 2 generation cost, and c_d denotes the direct-extraction proxy per page. Costs are in USD. The plotted thresholds are $N_B^* = G_B/c_d$ and $N_T^* = G_T/c_d$. All calculations use unrounded inputs; table entries are rounded for presentation.

Document	G_B	G_T	c_d	N_B^*	N_T^*
`id_01`	0.3719	0.7999	0.033570	11.08	23.83
`id_02`	0.8225	0.8989	0.047457	17.33	18.94
`id_03`	0.9147	0.5015	0.091061	10.05	5.51
`id_04`	0.7809	0.0975	0.003773	206.97	25.86
`id_05`	1.8783	0.3992	0.002446	767.87	163.22
`id_06`	1.0945	0.1044	0.031143	35.15	3.35
`id_07`	0.9205	0.1159	0.091046	10.11	1.27
`id_08`	1.0076	0.1886	0.155715	6.47	1.21
`id_09`	1.0940	0.0914	0.122319	8.94	0.75
`id_10`	1.2458	0.6546	0.074382	16.75	8.80
`id_11`	0.4672	1.0474	0.029225	15.99	35.84
`id_12`	0.7549	0.1225	0.041928	18.00	2.92
`id_13`	0.9768	0.1479	0.004334	225.39	34.12
`id_14`	0.8865	0.3746	0.025429	34.86	14.73
`id_15`	0.8215	0.2688*	0.061035	13.46	4.40*
`id_16`	1.2158	0.0917	0.030517	39.84	3.00
`id_17`	1.5792	0.0738	0.036328	43.47	2.03
`id_18`	0.9938	0.0752	0.044559	22.30	1.69
`id_19`	1.1409	0.0774	0.047608	23.96	1.63
`id_20`	1.0232	0.3844	0.022113	46.27	17.38
`id_21`	1.0189	0.2494	0.036799	27.69	6.78
`id_22`	0.9619	0.2614	0.058593	16.42	4.46
`id_23`	0.3400	1.5832	0.039134	8.69	40.46

Table C.1: Generation-only construction costs and continuous break-even page counts, excluding reference generation and scoring. An asterisk denotes the legacy generation-price fallback. For `id_06` and `id_14`, c_d uses separately supplied full-PDF costs.

These estimates assume reuse without reconstruction and do not adjust costs for differing accuracy or execution-failure rates.